JSW · COLD ROLLING · CPL-2

# CPL-2 Scheduler

## From scratch to deploy

A complete guide to the coil scheduling web app: the rules taken from the CPL-2 schedule deck, how the app applies them, every line of code, and how to put it online for free on Render with MongoDB Atlas, so it runs on any phone or computer.

| | |
|---|---|
| Prepared by | Aditi Singh, CSE (AIML), Vishwakarma Institute of Technology, Pune |
| Date | 15 October 2026 |
| Sources | CPL-2_Schedule_PPT.pptx (16 slides), CPL2_HR_STOCK_FOR_SCHEDULING.xls (Hrstock Report) |
| Stack | Node.js 20 + Express, MongoDB Atlas (free M0), plain HTML/CSS/JavaScript, SheetJS for .xls files |
| Hosting | Render free web service, code on GitHub |

### Contents


1. What you are building
2. The scheduling rules
3. How the app applies the rules
4. Tools to install
5. Build the project step by step
6. Run and test on your Mac
7. Create the MongoDB Atlas database
8. Put the code on GitHub
9. Deploy on Render
10. Use it on any device
11. Updating the app
12. Troubleshooting

Appendix A: API reference

Appendix B: complete source code

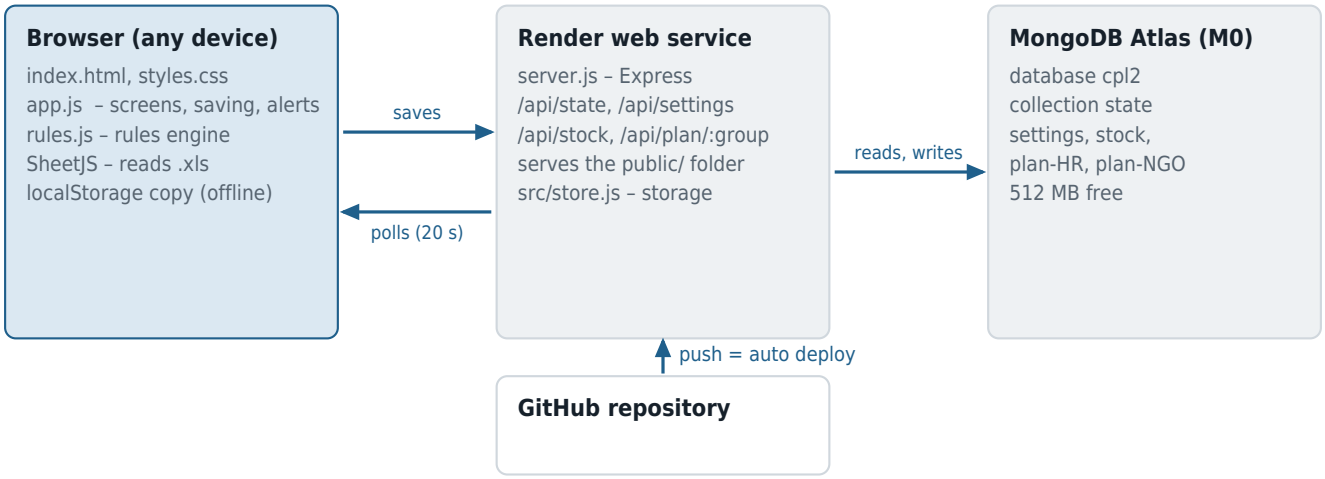
# 1. What you are building

A web app for the CPL-2 planner. It reads the HR stock report, applies the data-prep steps from the deck, checks every coil against the coil rolled before it, and arranges both schedules automatically. All rule values can be changed in the app, and the changes are shared by every device that opens the link.

## Features

- **Load HR stock file**: reads the .xls / .xlsx export of the Hrstock Report in the browser.
- **Data prep**: deck steps 1–14 plus your filters, with row counts for each step.
- **HR schedule** (HRPO + HRSPO) and a separate **NGO FP report**, each with Auto-arrange, sorting and ↑↓ to move coils.
- **Rules & settings**: every limit editable, customer and grade pick lists, editable NGO welding matrix.
- **Notifications**: bell with alert list, pop-ups in the page, and optional device notifications.
- **Shared state**: settings, stock file and coil order saved on the server, so phone and laptop show the same thing.
- **Editor key**: only people with the key can change the shared rules.

## Architecture




The rules run in the browser, so the page reacts instantly when a value changes. The server only stores and shares data. Each page asks the server for changes every 20 seconds and applies anything newer, so a rule changed on a laptop shows up on a phone within 20 seconds. If two people change the same thing at once, the last save wins.

> **Why Render and not Vercel?** The app keeps a small Express server running that the pages poll. Render runs a normal Node server on its free plan with no code changes. Vercel runs serverless functions, so the server would need to be rewritten as separate functions. Render is the simpler free choice here.

# 2. The scheduling rules

Everything below comes from the CPL-2 schedule deck (slides 1–15), plus the updated values you set. Every number in this section can be changed in the app under **Rules & settings**.

## Part A – Preparing the data (deck steps 1–14)

| Step | What the deck says | How the app does it |
|---|---|---|
| 1 | Delete the 1st column and row of the report. | Finds the row containing “HR Coil No” and uses it as the header. |
| 2 | Filter Product: HRPO / HRSPO / NGOFP. | Keeps Product HRPO, HRSPO or NGO FP. |
| 3–4 | Select Act Path and Prev Unit, select only zero. | Keeps rows where both are 0. |
| 5–6 | Copy to the Plan sheet, delete unwanted columns. | Keeps only the columns needed for scheduling. |
| 7–8 | Status: remove TD and TU, only TA and TW. | Keeps the statuses ticked in settings (default TA, TW). |
| 9 | Order status: delete all CLOSED. | Removes Order_status = CLOSED. |
| 10–13 | Receiving remarks APL7 → APL7 sheet, delete from PLAN. | Moves Receiving Remarks = APL7 to the APL7 list. |
| Filters | (new) | Thickness range, customers and grades from settings. |
| 14 | Final schedule data. | Splits into the HR schedule and the NGO FP report. |

## Part B – HRPO / HRSPO rules between two coils in a row

| Rule | Deck | Your value (default) |
|---|---|---|
| Thickness jump, thinner gauge | 0.4 mm in 1.6–2.0 mm | 0.8 mm |
| Thickness jump, thicker gauge | 1 mm in 2–6 mm | 1 mm for 4 mm and above |
| Thicker gauge starts at | – | 4 mm |
| Width change, thinner gauge | < 150 mm below 2.0 mm | 180 mm |
| Width change, thicker gauge | < 250–300 mm | 300 mm (check above 250 mm) |
| Critical TDC | JVPST01C00, JVPFB60AJS, JVPTR14AJS, JVPTR15AJS, JVPTR13AJS not continuous above 4.5 mm | Same list and 4.5 mm |
| Same thickness together | – | Strict: coils of one thickness run one after another |
| Range of thickness | 1.6–6 mm | 1.6–6 mm |
| Start thickness / width | – | Blank: thinnest coil, widest first |
| Age | Above 72 hours only | 72 h |
| Status | TA and TW | TA, TW |

## Part C – NGO FP report

NGO FP is scheduled separately. Neighbouring coils may change width by at most **150 mm**, and the silicon grades must be allowed to weld together. Rows are the coil rolled first, columns the coil after it.

| First ↓ Next → | LSQ2L 0.45% | MSQ1 0.9% | MSQ2 1.3% | MSQ3 1.55% | HSQ1 1.7% | HSQ2 2.0% | HSQ3 2.2% | HSQ5 2.6% |
|---|---|---|---|---|---|---|---|---|
| LSQ2L (≤0.45%) | YES | YES | YES | YES | YES | NO | NO | NO |
| MSQ1 (≤0.9%) | YES | YES | YES | YES | YES | NO | NO | NO |
| MSQ2 (≤1.3%) | YES | YES | YES | YES | YES | YES | NO | NO |
| MSQ3 (≤1.55%) | YES | YES | YES | YES | YES | YES | YES | NO |
| HSQ1 (≤1.7%) | YES | YES | YES | YES | YES | YES | YES | YES |
| HSQ2 (≤2.0%) | YES | YES | YES | YES | YES | YES | NO | NO |
| HSQ3 (≤2.2%) | NO | NO | NO | YES | YES | NO | NO | NO |
| HSQ5 (≤2.6%) | NO | NO | NO | NO | YES | NO | NO | NO |

> Points still to confirm with the planner: the 250–300 mm width range, grades LSQ2 and HRB that are not in the matrix, remarks typed “APL 7” instead of APL7, and whether NGO FP needs a thickness limit.

# 3. How the app applies the rules

## Checking a sequence

For every coil in the order, the app compares it with the coil before it (file `public/rules.js`, functions `pairIssues` and `evaluate`):

- **Thinner or thicker gauge**: if the thinner of the two coils is below “thicker gauge starts at” (4 mm), the thinner-gauge limits apply.
- **Thickness jump** above the limit → **Break**.
- **Width change** above the limit → Break. For thicker gauge, a change between the check value and the limit → **Check**.
- **Critical TDC**: both coils above 4.5 mm and both TDCs in the list → Break.
- **NGO FP**: matrix says NO → Break; grade not in matrix → Check; width jump above 150 mm → Break.
- **Same thickness**: a thickness that already appeared earlier, after a different thickness → Break.
- **Age**: coils aged 72 h or less are marked **Held** and moved to the end by Auto-arrange.

## Auto-arrange

1. Set aside held coils (too fresh).
2. Group the remaining coils by thickness (one block per thickness).
3. Start with the block closest to the start thickness (or the thinnest). Inside it, start with the coil closest to the start width (or the widest).
4. Inside a block, repeatedly pick the next coil with the lowest cost: rule breaks cost the most, then checks, then a change of TDC/grade, then the width change.
5. When a block is finished, move to the nearest thickness not yet used and pick its first coil the same way.
6. Add the held coils at the end.

On the sample stock file this turns 182 rule breaks in sheet order into 25 (HR), and 5 into 0 (NGO FP). The breaks left are width changes inside a thickness block that cannot be avoided while every thickness stays together.

## Saving and sharing

- Every change is saved in the browser first (localStorage), so the page works even if the server is asleep.
- Then it is sent to the server: `PUT /api/settings`, `/api/stock` or `/api/plan/HR|NGO`.
- Each page calls `GET /api/state` every 20 s and applies anything newer than what it has.
- When `ADMIN_KEY` is set on the server, saving needs that key. People without it can look but not change shared data.

## Notifications

- The bell lists current alerts: rule breaks, held coils, APL7 remarks typed differently, and changes made on another device.
- A pop-up appears in the page when a new alert arrives.
- With **Device notifications** ticked (the browser asks for permission once), an alert also appears as a system notification while the page is open in a background tab.
- Notifications cannot arrive when every tab with the app is closed. That would need a push service, which is outside this free setup.

# 4. Tools to install

Everything here is free. The commands are for macOS Terminal.

| Tool | Why | Get it |
|---|---|---|
| Node.js 20 or newer (LTS) | Runs the server and the tests | nodejs.org → download the LTS installer for macOS |
| Git | Sends the code to GitHub | Run git --version; macOS offers to install it if missing |
| VS Code (optional) | Editing the files | code.visualstudio.com |
| GitHub account | Stores the code; Render deploys from it | github.com |
| MongoDB Atlas account | Free database for shared settings | mongodb.com/cloud/atlas |
| Render account | Free hosting | render.com → sign up with GitHub |

### Check the installs:

```
node -v      # v20.x or newer
npm -v
git --version
```

# 5. Build the project step by step

Create the folder and the files below. The full code of every file is in Appendix B; type or copy each one into the file of the same name.

## 5.1 Create the folder and install packages

```
mkdir cpl2-scheduler && cd cpl2-scheduler
mkdir -p src public test
npm init -y
npm install express mongodb dotenv
```

Then replace the generated `package.json` with the one in Appendix B (it adds the `start`, dev and `test` scripts).

## 5.2 Project structure

```
cpl2-scheduler/
├── package.json          project info, dependencies, npm scripts
├── server.js             Express server + API
├── src/
│   └── store.js          MongoDB or local JSON file storage
├── public/               everything the browser loads
│   ├── index.html        page layout
│   ├── styles.css        look and feel, light and dark mode
│   ├── rules.js          rules engine (also used by the tests)
│   ├── app.js            screens, saving, sync, notifications
│   └── sample-stock.json optional: sample stock shown on first open
├── test/
│   └── rules.test.js     checks the rules with node --test
├── render.yaml           Render settings (optional blueprint)
├── .env.example          example environment variables
├── .gitignore
└── README.md
```

## 5.3 What each file does

- **server.js**: starts Express, serves the public folder, and offers the five API routes. Checks the editor key on every save.
- **src/store.js**: one small interface (getAll, put). With MONGODB_URI it uses MongoDB; without it, data/state.json.
- **public/rules.js**: all rule logic in pure functions: data prep, pair checks, sequence check, Auto-arrange. No screen code, so Node can test it.
- **public/app.js**: draws the four tabs, handles clicks and inputs, reads .xls files, saves and syncs with the server, raises notifications.
- **public/index.html / styles.css**: the page skeleton and the design, including phone layout and dark mode.
- **test/rules.test.js**: eight checks of the rules (0.8 mm, 1 mm, 180 mm, 150 mm, matrix, critical TDC, thickness blocks).

## 5.4 The sample stock file (optional)

`public/sample-stock.json` lets the app open with your stock sheet already loaded. It is too long to print, so it is not in Appendix B. Without it, the app opens empty and you click **Load HR stock file** once; the file is then saved on the server and every device sees it.

# 6. Run and test on your Mac

## 6.1 Run the tests

```
npm test
```

Expected end of the output:

```
# tests 8
# pass 8
# fail 0
```

## 6.2 Start the app

```
npm start
# CPL-2 Scheduler on http://localhost:3000 (storage: file)
```

Open **http://localhost:3000**. Without MongoDB the data is saved in `data/state.json`.

1. Click **Load HR stock file** and choose CPL2_HR_STOCK_FOR_SCHEDULING.xls.
2. Open **Data prep**: 270 rows → 60 to APL7 → 143 coils for scheduling.
3. Open **HR schedule**, click **Auto-arrange**: 133 coils in 29 thickness blocks, 25 rule breaks.
4. Open **Rules & settings**, change “Width change, thinner gauge” to 100: the HR count of breaks goes up straight away.
5. Open a second browser (Safari and Chrome) on the same address: after up to 20 s it shows the same rules and order.

## 6.3 Try the editor key

```
ADMIN_KEY=test123 npm start
```

Rules & settings now shows an **Editor key** box. Without the key, changes stay on that device only.

Stop the server with **Ctrl + C**.

# 7. Create the MongoDB Atlas database

1. Sign in at mongodb.com/cloud/atlas and create a project (for example **CPL2**).
2. Click **Build a Database** (or **Create**) and choose **Free / M0**. Pick a region close to you, for example AWS Mumbai (ap-south-1) if it is offered. Create the deployment.
3. **Database user**: choose username and password, for example `cpl2app`, and generate a password. Save it; you need it in step 6.
4. **Network Access**: add IP address `0.0.0.0/0` (allow from anywhere). Render's free plan has no fixed IP, so this is needed.
5. Click **Connect → Drivers → Node.js** and copy the connection string.
6. Put your password into it. This full string is your `MONGODB_URI`.

```
mongodb+srv://cpl2app:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority
```

> If the password has special characters such as @ : / ? #, they must be URL-encoded in the string (for example @ becomes %40), or generate a password with letters and digits only.

## Optional: test it on your Mac

```
cp .env.example .env
# edit .env and paste the string after MONGODB_URI=
npm start
# CPL-2 Scheduler on http://localhost:3000 (storage: mongodb)
```

After the first save, Atlas → Browse Collections shows database `cpl2`, collection `state` with documents settings, stock, plan-HR and plan-NGO.

> Never commit .env to GitHub. It is already listed in .gitignore.

# 8. Put the code on GitHub

1. On github.com click **New repository**, name it `cpl2-scheduler`, choose Private or Public, and create it without a README.
2. In Terminal, inside the project folder:

```
git init
git add .
git commit -m "CPL-2 scheduler"
git branch -M main
git remote add origin https://github.com/<your-username>/cpl2-scheduler.git
git push -u origin main
```

If GitHub asks for a password, use a personal access token (GitHub → Settings → Developer settings → Personal access tokens), or sign in with GitHub Desktop.

Check on github.com that you see server.js, public/ and src/, and that **node_modules** and **.env** are not there.

# 9. Deploy on Render

1. Sign in at render.com with your GitHub account.
2. Click **New → Web Service** and connect the `cpl2-scheduler` repository.
3. Fill in the settings as in the table below.
4. Under **Environment Variables**, add MONGODB_URI and ADMIN_KEY.
5. Click **Create Web Service** (or **Deploy**). The first build takes a few minutes.
6. In the **Logs** tab, wait for `CPL-2 Scheduler on … (storage: mongodb)`.
7. Open the address shown at the top, for example `https://cpl2-scheduler.onrender.com`.

| Setting | Value |
|---|---|
| Name | cpl2-scheduler (becomes part of the address) |
| Language / Runtime | Node |
| Branch | main |
| Build command | npm install |
| Start command | npm start |
| Instance type | Free |
| Health check path (Advanced) | /api/health |
| Environment variable MONGODB_URI | your Atlas connection string |
| Environment variable ADMIN_KEY | a secret word for editors, e.g. a long random phrase |
| Environment variable NODE_VERSION | 20 (optional) |

## Check that it works

```
https://<your-app>.onrender.com/api/health
# {"ok":true,"store":"mongodb"}
```

If it says `"store":"file"`, MONGODB_URI is missing and data would be lost on every restart.

> **Alternative:** render.yaml in the project is a Render Blueprint. Choose **New → Blueprint**, pick the repository, and Render reads the settings from the file; it then asks for MONGODB_URI and ADMIN_KEY.

## Free plan limits

- The service sleeps after 15 minutes with no visits. The next visit wakes it in about a minute; the page still opens from the device’s saved copy and catches up when the server answers.
- 750 free instance hours per month per workspace, enough for one service running all month.
- Files written on the server are lost on restart, which is why the data goes to MongoDB Atlas.
- Atlas M0 gives 512 MB, far more than this app needs.

# 10. Use it on any device

- **Share the link** with the team. Anyone with it can view; only people with the editor key can change shared data.
- **Editor key**: Rules & settings → Editor key. It is stored on that device only, once per device.
- **Phone**: open the link in Safari or Chrome. Safari: Share → Add to Home Screen. Chrome on Android: ⋮ → Add to Home screen. It then opens like an app.
- **Device notifications**: Rules & settings → Notifications → tick Device notifications and allow. On iPhone this needs the app added to the Home Screen first (iOS 16.4 or later).
- **New stock file**: click Load HR stock file on any device; everyone gets it within 20 s.
- **Daily flow**: load the new HR stock → check Data prep → Auto-arrange both schedules → fix remaining breaks with ↑↓ → the order is saved for everyone.

# 11. Updating the app

- **Changing a rule value**: no code change needed. Do it in Rules & settings.
- **Changing the default values** for new installs: edit DEFAULTS at the top of public/rules.js. Existing saved settings stay as they are; use Reset to default rules to apply the new defaults.
- **Changing code**: edit, run `npm test`, then push. Render redeploys automatically:

```
git add .
git commit -m "Describe the change"
git push
```

# 12. Troubleshooting

| What you see | Cause and fix |
|---|---|
| Header says “Offline: saved on this device” | Server asleep or unreachable. Wait a minute and reload. Check the Render dashboard. |
| “Enter the editor key …” | ADMIN_KEY is set. Type the same key in Rules & settings → Editor key. |
| /api/health shows "store":"file" | MONGODB_URI not set on Render. Add it under Environment and redeploy. |
| Render log: “Could not start: bad auth” | Wrong user or password in MONGODB_URI, or special characters not URL-encoded. |
| Render log: connection timed out | Atlas Network Access is missing 0.0.0.0/0. |
| “No “HR Coil No” column found” | Load the Hrstock Report export itself; the header row must contain HR Coil No. |
| “The spreadsheet reader did not load” | SheetJS loads from cdnjs. Check the internet connection and reload. |
| File too big to share | Very large stock files may exceed the 5 MB request limit set in server.js (express.json limit). |
| Device notifications do not appear | Allow notifications for the site in the browser settings; on iPhone add to Home Screen first. |
| npm test fails after a change | Read the failing test name; it says which rule no longer behaves as expected. |

# Appendix A: API reference

| Method and path | Body | Returns |
|---|---|---|
| GET /api/health | – | { ok: true, store: "mongodb" \| "file" } |
| GET /api/state | – | { requiresKey, state: { settings, stock, plan-HR, plan-NGO } }, each { data, updatedAt } |
| PUT /api/settings | settings object | { data, updatedAt } |
| PUT /api/stock | { name, cols, rows } | { data, updatedAt } |
| PUT /api/plan/HR or /NGO | { ids: ["HR coil no", …] } | { data, updatedAt } |

PUT requests need the header `x-admin-key` when ADMIN_KEY is set; otherwise the server answers 401.

## Environment variables

| Name | Required | Meaning |
|---|---|---|
| MONGODB_URI | On Render: yes | Atlas connection string. Empty = local file storage. |
| MONGODB_DB | No | Database name, default cpl2. |
| ADMIN_KEY | Recommended | Editor key needed to save shared changes. |
| PORT | No | Render sets it automatically; default 3000. |

## Sources


- Render, “Deploy for Free”: https://render.com/docs/free
- OneUptime, “How to Set Up MongoDB Atlas Free Tier Cluster” (2026): https://oneuptime.com/blog/post/2026-03-31-mongodb-atlas-free-tier-setup/view
- CPL-2_Schedule_PPT.pptx and CPL2_HR_STOCK_FOR_SCHEDULING.xls (JSW)

# Appendix B: complete source code

Every file of the project, exactly as tested. Create each file with the path shown and copy its contents. Lines are at most 140 characters, so nothing is wrapped.

**package.json** · 18 lines

```
{
  "name": "cpl2-scheduler",
  "version": "1.0.0",
  "description": "CPL-2 coil scheduling rulebook and scheduler (HRPO/HRSPO and NGO FP)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js",
    "test": "node --test"
  },
  "engines": { "node": ">=20" },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "mongodb": "^6.12.0"
  }
}
```

**.gitignore** · 4 lines

```
node_modules/
data/
.env
```

**.env.example** · 8 lines

```
# Copy this file to .env for local development.
# Leave MONGODB_URI empty to store data in ./data/state.json instead.
MONGODB_URI=
MONGODB_DB=cpl2
# Optional: if set, only people who enter this key in Rules & settings can save changes.
ADMIN_KEY=
PORT=3000
```

**render.yaml** · 16 lines

```
services:
  - type: web
    name: cpl2-scheduler
    runtime: node
    plan: free
    buildCommand: npm install
    startCommand: npm start
    healthCheckPath: /api/health
    envVars:
      - key: NODE_VERSION
        value: "20"
      - key: MONGODB_URI
        sync: false
      - key: ADMIN_KEY
        sync: false
```

**server.js** · 70 lines

```
/*
 * CPL-2 Scheduler server.
 * Serves the web app from ./public and a small JSON API that keeps
 * settings, the stock file and the saved orders the same on every device.
 */
require("dotenv").config();
const express = require("express");
const path = require("path");
const { createStore } = require("./src/store");

const PORT = process.env.PORT || 3000;
const ADMIN_KEY = process.env.ADMIN_KEY || ""; // if set, saving needs this key

async function main() {
  const store = await createStore();
  const app = express();
  app.use(express.json({ limit: "5mb" }));

  // Only people with the editor key can change shared data (when ADMIN_KEY is set).
  const canWrite = (req, res, next) => {
    if (!ADMIN_KEY || req.get("x-admin-key") === ADMIN_KEY) return next();
    res.status(401).json({ error: "Editor key missing or wrong" });
  };

  app.get("/api/health", (req, res) => res.json({ ok: true, store: store.kind }));

  // Everything the page needs in one call. The page polls this to pick up changes.
  app.get("/api/state", async (req, res, next) => {
    try {
      const all = await store.getAll();
      res.json({ requiresKey: !!ADMIN_KEY, state: all });
```

```
    } catch (e) { next(e); }
  });

  app.put("/api/settings", canWrite, async (req, res, next) => {
    try {
      if (!req.body || typeof req.body !== "object") return res.status(400).json({ error: "Settings must be an object" });
      res.json(await store.put("settings", req.body));
    } catch (e) { next(e); }
  });

  app.put("/api/stock", canWrite, async (req, res, next) => {
    try {
      const b = req.body || {};
      if (!Array.isArray(b.cols) || !Array.isArray(b.rows)) return res.status(400).json({ error: "Stock needs cols and rows" });
      res.json(await store.put("stock", { name: String(b.name || "stock"), cols: b.cols, rows: b.rows }));
    } catch (e) { next(e); }
  });

  app.put("/api/plan/:group", canWrite, async (req, res, next) => {
    try {
      const g = req.params.group;
      if (g !== "HR" && g !== "NGO") return res.status(400).json({ error: "Group must be HR or NGO" });
      const ids = Array.isArray(req.body && req.body.ids) ? req.body.ids.map(String) : [];
      res.json(await store.put("plan-" + g, { ids }));
    } catch (e) { next(e); }
  });

  app.use(express.static(path.join(__dirname, "public")));

  app.use((err, req, res, next) => {
    console.error(err);
    res.status(500).json({ error: "Server error" });
  });

  app.listen(PORT, () => console.log(`CPL-2 Scheduler on http://localhost:${PORT} (storage: ${store.kind})`));
}

main().catch(e => { console.error("Could not start:", e.message); process.exit(1); });
```

**src/store.js** · 54 lines

```
/*
 * Storage for the shared app state: settings, stock file, and the two saved orders.
 * Uses MongoDB when MONGODB_URI is set (production on Render),
 * otherwise a JSON file in ./data (local development).
 */
const fs = require("fs");
const path = require("path");

const KEYS = ["settings", "stock", "plan-HR", "plan-NGO"];
```

```
function fileStore(file) {
  fs.mkdirSync(path.dirname(file), { recursive: true });
  const read = () => { try { return JSON.parse(fs.readFileSync(file, "utf8")); } catch { return {}; } };
  return {
    kind: "file",
    async getAll() { return read(); },
    async put(key, data) {
      const all = read();
      all[key] = { data, updatedAt: Date.now() };
      fs.mkdirSync(path.dirname(file), { recursive: true });
      fs.writeFileSync(file, JSON.stringify(all));
      return all[key];
    }
  };
}

async function mongoStore(uri) {
  const { MongoClient } = require("mongodb");
  const client = new MongoClient(uri);
  await client.connect();
  const col = client.db(process.env.MONGODB_DB || "cpl2").collection("state");
  return {
    kind: "mongodb",
    async getAll() {
      const docs = await col.find({ _id: { $in: KEYS } }).toArray();
      const out = {};
      docs.forEach(d => { out[d._id] = { data: d.data, updatedAt: d.updatedAt }; });
      return out;
    },
    async put(key, data) {
      const doc = { data, updatedAt: Date.now() };
      await col.updateOne({ _id: key }, { $set: doc }, { upsert: true });
      return doc;
    }
  };
}

async function createStore() {
  if (process.env.MONGODB_URI) return mongoStore(process.env.MONGODB_URI);
  return fileStore(path.join(__dirname, "..", "data", "state.json"));
}

module.exports = { createStore, KEYS };
```

**public/index.html** · 84 lines

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1,viewport-fit=cover">
<meta name="theme-color" content="#1f5f8b">
<title>CPL-2 Scheduling Rulebook</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=IBM+Plex+Mono:wght@400;500&display=swap">
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans+Condensed:wght@500;600;700&display=swap">
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:wght@400;500;600&display=swap">
<link rel="stylesheet" href="styles.css">
</head>
<body>

<header class="top"><div class="wrap">
  <div class="hgroup">
    <div class="label">JSW · Cold Rolling · CPL-2</div>
    <h1>CPL-2 Scheduling Rulebook</h1>
    <div class="src">
      <span class="chip" id="srcName" title="Stock file in use">CPL2_HR_STOCK_FOR_SCHEDULING.xls</span>
      <label class="btn" for="file">Load HR stock file</label>
      <input id="file" type="file" accept=".xls,.xlsx,.csv" hidden>
    </div>
  </div>
  <div class="hright">
    <span class="sync" id="sync"><i></i><span>Saved on this device</span></span>
    <button class="btn bell" id="bellBtn" aria-label="Notifications">Notifications<b id="bellCount" hidden>0</b></button>
  </div>
</div></header>

<nav class="tabs" aria-label="Sections"><div class="wrap" role="tablist">
  <button role="tab" data-tab="hr" aria-selected="true">HR schedule<span id="cHR"></span></button>
  <button role="tab" data-tab="ngo" aria-selected="false">NGO FP report<span id="cNGO"></span></button>
  <button role="tab" data-tab="rules" aria-selected="false">Rules &amp;amp; settings</button>
  <button role="tab" data-tab="prep" aria-selected="false">Data prep</button>
</div></nav>

<main>
  <section class="panel" id="p-hr"><div class="wrap"></div></section>
  <section class="panel" id="p-ngo" hidden><div class="wrap"></div></section>

  <section class="panel" id="p-rules" hidden><div class="wrap">
    <div class="phead"><div>
      <div class="label">Every value is editable</div>
      <h2>Rules &amp;amp; settings</h2>
      <p>Change any limit and both schedules re-check at once. Settings save automatically and stay the same
        the next time you open the page, on any device.</p>
    </div></div>
```

```
    <div class="sgrid" id="settingsForm"></div>
    <div class="savebar">
      <span id="saveState" class="sync"><i></i><span>All changes saved</span></span>
      <span style="display:flex;gap:8px;flex-wrap:wrap">
        <button class="btn danger" id="resetBtn">Reset to default rules</button>
      </span>
    </div>
  </div></section>

  <section class="panel" id="p-prep" hidden><div class="wrap">
    <div class="phead"><div>
      <div class="label">Deck steps 1–14, plus your filters</div>
      <h2>Preparing the schedule data</h2>
      <p>How the raw Hrstock Report becomes the coils on the two schedules. Counts are for the stock file in use.</p>
    </div></div>
    <div class="funnel" id="funnel"></div>
    <p class="note" id="aplNote" hidden></p>
  </div></section>
</main>

<div class="shade" id="shade" hidden></div>
<aside class="drawer" id="drawer" hidden aria-label="Notifications">
  <header><h3>Notifications</h3><span style="display:flex;gap:8px"><button class="btn" id="markRead">Mark all read</button>
  <button class="btn" id="closeDrawer" aria-label="Close">Close</button></span></header>
  <div class="nlist" id="nlist"></div>
</aside>
<div class="toast" id="toast" hidden role="status"></div>

<script src="https://cdnjs.cloudflare.com/ajax/libs/xlsx/0.18.5/xlsx.full.min.js"></script>
<script src="rules.js"></script>
<script src="app.js"></script>
</body>
</html>
```

**public/styles.css** · 196 lines

```
/* Layout: plant-floor scheduling tool. Header with source + alerts, tab rail,
   one working panel at a time; data set in mono like a mill sheet. */
:root{
  --bg:#eef1f3; --surface:#ffffff; --sunk:#e4e8eb; --ink:#16202a; --muted:#56636f; --line:#d0d7dd;
  --accent:#1f5f8b; --accent-ink:#ffffff; --accent-soft:#dbe8f2; --heat:#a65a0c; --heat-soft:#fbecd9;
  --ok:#2c7a4b; --ok-soft:#dcf0e3; --bad:#b3261e; --bad-soft:#f8dfdc; --shade:rgba(16,24,32,.35);
  --f-display:"IBM Plex Sans Condensed","Arial Narrow",system-ui,sans-serif;
  --f-body:"IBM Plex Sans",system-ui,-apple-system,"Segoe UI",sans-serif;
  --f-mono:"IBM Plex Mono",ui-monospace,"SFMono-Regular",Menlo,Consolas,monospace;
}
@media (prefers-color-scheme:dark){:root:not([data-theme="light"]){
  --bg:#10151a; --surface:#172028; --sunk:#1d2730; --ink:#e2e8ed; --muted:#97a4af; --line:#2b3742;
  --accent:#71b2df; --accent-ink:#0d1a24; --accent-soft:#1b3346; --heat:#e8a052; --heat-soft:#3a2914;
  --ok:#62c48a; --ok-soft:#173326; --bad:#f07f76; --bad-soft:#3d1c1a; --shade:rgba(0,0,0,.55); color-scheme:dark}}
:root[data-theme="dark"]{
```

```
  --bg:#10151a; --surface:#172028; --sunk:#1d2730; --ink:#e2e8ed; --muted:#97a4af; --line:#2b3742;
  --accent:#71b2df; --accent-ink:#0d1a24; --accent-soft:#1b3346; --heat:#e8a052; --heat-soft:#3a2914;
  --ok:#62c48a; --ok-soft:#173326; --bad:#f07f76; --bad-soft:#3d1c1a; --shade:rgba(0,0,0,.55); color-scheme:dark}
*{box-sizing:border-box}
html{-webkit-text-size-adjust:100%}
body{margin:0}
img{max-width:100%}
[hidden]{display:none!important}
body{background:var(--bg);color:var(--ink);font-family:var(--f-body);font-size:15px;line-height:1.5}
.wrap{max-width:1200px;margin:0 auto;padding-left:max(16px,env(safe-area-inset-left,0px));
padding-right:max(16px,env(safe-area-inset-right,0px))}
h1,h2,h3{font-family:var(--f-display);text-wrap:balance;margin:0;line-height:1.15}
h1{font-size:clamp(1.7rem,4.5vw,2.6rem);font-weight:700}
h2{font-size:1.5rem;font-weight:600}
h3{font-size:1.05rem;font-weight:600}
p{margin:0;max-width:70ch}
.mono,code{font-family:var(--f-mono);font-variant-numeric:tabular-nums}
code{font-size:.88em;background:var(--sunk);padding:.05em .35em;border-radius:3px}
.label{font-family:var(--f-display);font-size:.76rem;font-weight:600;letter-spacing:.09em;text-transform:uppercase;color:var(--muted)}
.muted{color:var(--muted)}
button,select,input,textarea{font:inherit;color:inherit}
:focus-visible{outline:2px solid var(--accent);outline-offset:2px}
.btn{cursor:pointer;border:1px solid var(--line);background:var(--surface);color:var(--ink);padding:7px 12px;border-radius:4px;
font-size:.88rem;white-space:nowrap;display:inline-flex;align-items:center;gap:6px}
.btn:hover{border-color:var(--accent)}
.btn.primary{background:var(--accent);color:var(--accent-ink);border-color:var(--accent)}
.btn.danger{color:var(--bad)}
.btn:disabled{opacity:.5;cursor:not-allowed}
.chip{display:inline-flex;align-items:center;gap:6px;font-family:var(--f-mono);font-size:.76rem;padding:3px 9px;border-radius:999px;
background:var(--sunk);border:1px solid var(--line);max-width:100%;overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
.pill{display:inline-flex;align-items:center;gap:5px;font-size:.76rem;font-weight:600;padding:2px 9px;border-radius:999px;
white-space:nowrap}
.pill.ok{background:var(--ok-soft);color:var(--ok)}
.pill.warn{background:var(--heat-soft);color:var(--heat)}
.pill.bad{background:var(--bad-soft);color:var(--bad)}
.pill.info{background:var(--sunk);color:var(--muted)}

/* header */
header.top{background:var(--surface);border-bottom:1px solid var(--line)}
header.top .wrap{padding-block:22px 18px;display:flex;flex-wrap:wrap;gap:16px 24px;align-items:flex-end;justify-content:space-between}
.hgroup{display:grid;gap:6px;min-width:0}
.src{display:flex;gap:8px;align-items:center;flex-wrap:wrap}
.hright{display:flex;gap:8px;align-items:center;flex-wrap:wrap}
.sync{font-size:.8rem;color:var(--muted);display:inline-flex;align-items:center;gap:6px}
.sync i{width:8px;height:8px;border-radius:50%;background:var(--muted);display:inline-block}
.sync.on i{background:var(--ok)}
.sync.local i{background:var(--heat)}
.bell{position:relative}
.bell b{position:absolute;top:-6px;right:-6px;min-width:18px;height:18px;padding:0 5px;border-radius:9px;background:var(--bad);
color:var(--surface);font-size:.7rem;display:grid;place-items:center;font-family:var(--f-mono)}

/* tabs */
```

```
nav.tabs{position:sticky;top:env(safe-area-inset-top,0px);z-index:6;background:var(--bg);border-bottom:1px solid var(--line)}
nav.tabs .wrap{display:flex;gap:2px;overflow-x:auto;padding-block:6px}
nav.tabs button{border:0;background:transparent;white-space:nowrap;cursor:pointer;color:var(--muted);font-family:var(--f-display);
font-weight:600;font-size:.95rem;padding:8px 14px;border-radius:4px}
nav.tabs button:hover{color:var(--ink);background:var(--sunk)}
nav.tabs button[aria-selected="true"]{color:var(--ink);background:var(--surface);box-shadow:inset 0 -2px 0 var(--accent)}
nav.tabs button span{font-family:var(--f-mono);font-size:.75rem;color:var(--muted);margin-left:4px}
main .panel{padding-block:28px 56px}
.phead{display:flex;flex-wrap:wrap;gap:12px 24px;align-items:flex-end;justify-content:space-between;margin-bottom:18px}
.phead > div{display:grid;gap:6px;min-width:0}
.phead p{color:var(--muted)}

/* schedule */
.tiles{display:grid;grid-template-columns:repeat(5,minmax(0,1fr));gap:10px;margin-bottom:14px}
.tile{background:var(--surface);border:1px solid var(--line);border-radius:4px;padding:12px 14px;min-width:0}
.tile b{display:block;font-family:var(--f-display);font-size:1.7rem;font-weight:600;font-variant-numeric:tabular-nums;line-height:1.1}
.tile span{font-size:.8rem;color:var(--muted)}
.tile.bad b{color:var(--bad)} .tile.warn b{color:var(--heat)} .tile.ok b{color:var(--ok)}
.tools{display:flex;flex-wrap:wrap;gap:8px;align-items:center;margin-bottom:12px}
.tools .sep{flex:1}
.check{display:inline-flex;gap:6px;align-items:center;font-size:.86rem}
.applied .chip{white-space:normal;font-size:.74rem}
.applied{font-size:.82rem;color:var(--muted);margin-bottom:12px;display:flex;flex-wrap:wrap;gap:6px}
.tbl{border:1px solid var(--line);border-radius:4px;background:var(--surface);max-height:70vh;overflow:auto}
table{border-collapse:collapse}
table.seq{width:100%;font-size:.84rem}
table.seq th{position:sticky;top:0;background:var(--sunk);text-align:left;font-family:var(--f-display);font-weight:600;font-size:.76rem;
letter-spacing:.05em;text-transform:uppercase;color:var(--muted);padding:8px 10px;white-space:nowrap;z-index:1}
table.seq td{padding:6px 10px;border-top:1px solid var(--line);vertical-align:top}
table.seq td.n{text-align:right;white-space:nowrap;font-family:var(--f-mono);font-variant-numeric:tabular-nums}
table.seq td.cust{max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
table.seq tr.bad td:first-child{box-shadow:inset 3px 0 0 var(--bad)}
table.seq tr.warn td:first-child{box-shadow:inset 3px 0 0 var(--heat)}
table.seq tr.held td{opacity:.6}
table.seq tr.grp td{background:var(--accent-soft);padding:6px 10px;border-top:2px solid var(--accent);font-size:.8rem}
table.seq tr.grp b{font-family:var(--f-mono);font-weight:600;margin-right:10px}
table.seq tr.grp span{color:var(--muted)}
.mv{display:inline-flex;gap:2px}
.mv button{border:1px solid var(--line);background:var(--surface);color:var(--muted);width:26px;height:26px;border-radius:3px;
cursor:pointer;line-height:1;padding:0}
.mv button:hover{color:var(--ink);border-color:var(--accent)}
.why{display:flex;flex-direction:column;gap:3px;min-width:210px}
.why span{font-size:.76rem;color:var(--muted)}
.dv.b{color:var(--bad);font-weight:600}.dv.w{color:var(--heat);font-weight:600}

/* settings */
.sgrid{display:grid;grid-template-columns:repeat(2,minmax(0,1fr));gap:14px}
.card{background:var(--surface);border:1px solid var(--line);border-radius:4px;padding:16px 18px;display:grid;gap:12px;align-content:start;
min-width:0}
.card.wide{grid-column:1/-1}
.card header{display:grid;gap:4px}
.card header p{font-size:.86rem;color:var(--muted)}
```

```
.fields{display:grid;grid-template-columns:repeat(auto-fill,minmax(170px,1fr));gap:12px}
.field{display:grid;gap:4px;min-width:0}
.field label{font-size:.84rem;font-weight:500}
.inp{display:flex;align-items:center;border:1px solid var(--line);border-radius:4px;background:var(--bg);overflow:hidden}
.inp input{border:0;background:transparent;padding:7px 9px;width:100%;min-width:0;font-family:var(--f-mono);
font-variant-numeric:tabular-nums}
.inp span{padding:0 9px;font-size:.8rem;color:var(--muted);font-family:var(--f-mono)}
.field small{font-size:.76rem;color:var(--muted)}
.field small.changed{color:var(--heat)}
textarea{width:100%;border:1px solid var(--line);border-radius:4px;background:var(--bg);padding:8px 10px;font-family:var(--f-mono);
font-size:.84rem;resize:vertical}
.checks{display:flex;flex-wrap:wrap;gap:8px}
.checks label{display:inline-flex;gap:6px;align-items:center;border:1px solid var(--line);border-radius:4px;padding:5px 10px;
font-size:.86rem;background:var(--bg);cursor:pointer}
.picker{display:grid;gap:8px}
.picker .bar{display:flex;gap:8px;flex-wrap:wrap;align-items:center}
.picker input[type=search]{flex:1;min-width:160px;border:1px solid var(--line);border-radius:4px;background:var(--bg);padding:7px 10px}
.plist{max-height:300px;overflow:auto;border:1px solid var(--line);border-radius:4px;background:var(--bg)}
.plist label{display:grid;grid-template-columns:auto minmax(0,1fr) auto;gap:10px;align-items:center;padding:6px 10px;
border-top:1px solid var(--line);font-size:.86rem;cursor:pointer}
.plist label:first-child{border-top:0}
.plist label span:nth-child(2){overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
.plist label em{font-style:normal;font-family:var(--f-mono);font-size:.76rem;color:var(--muted)}
.plist label.off span:nth-child(2){color:var(--muted);text-decoration:line-through}
.scroll{overflow-x:auto;max-width:100%}
table.mx{font-size:.8rem;background:var(--surface)}
table.mx th,table.mx td{border:1px solid var(--line);padding:5px 6px;text-align:center}
table.mx th{font-family:var(--f-mono);font-weight:500;font-size:.74rem;white-space:nowrap}
table.mx td button{border:0;width:46px;padding:4px 0;border-radius:3px;cursor:pointer;font-family:var(--f-mono);font-size:.72rem;
font-weight:600}
table.mx td button.y{background:var(--ok-soft);color:var(--ok)}
table.mx td button.x{background:var(--bad-soft);color:var(--bad)}
.savebar{position:sticky;bottom:0;margin-top:18px;background:var(--surface);border:1px solid var(--line);border-radius:4px;
padding:10px 14px;display:flex;flex-wrap:wrap;gap:10px;align-items:center;justify-content:space-between;
padding-bottom:calc(10px + env(safe-area-inset-bottom,0px))}

/* prep */
.funnel{border:1px solid var(--line);border-radius:4px;background:var(--surface);overflow:hidden}
.stage{display:grid;grid-template-columns:80px minmax(0,1fr) 220px;gap:16px;padding:14px 18px;border-top:1px solid var(--line);
align-items:start}
.stage:first-child{border-top:0}
.stage .no{font-family:var(--f-mono);font-size:.8rem;color:var(--muted);padding-top:2px}
.stage p{font-size:.9rem;color:var(--muted)}
.stage .rule{margin-top:6px;font-family:var(--f-mono);font-size:.8rem}
.meter{display:grid;gap:5px;font-size:.8rem}
.meter .barx{height:8px;background:var(--sunk);border-radius:2px;overflow:hidden}
.meter .barx i{display:block;height:100%;background:var(--accent)}
.meter .nums{display:flex;justify-content:space-between;gap:8px;font-family:var(--f-mono);font-variant-numeric:tabular-nums}
.drop{color:var(--bad)}
.note{margin-top:14px;padding:10px 14px;border-left:3px solid var(--heat);background:var(--heat-soft);font-size:.88rem;max-width:none}

/* drawer + toast */
```

```
.shade{position:fixed;inset:0;background:var(--shade);z-index:20}
aside.drawer{position:fixed;top:0;right:0;bottom:0;width:min(420px,100%);background:var(--surface);border-left:1px solid var(--line);
z-index:21;display:flex;flex-direction:column;padding-top:env(safe-area-inset-top,0px);padding-right:env(safe-area-inset-right,0px);
padding-bottom:env(safe-area-inset-bottom,0px)}
aside.drawer header{display:flex;justify-content:space-between;align-items:center;padding:16px 18px;border-bottom:1px solid var(--line)}
.nlist{overflow:auto;flex:1}
.nitem{display:grid;grid-template-columns:10px minmax(0,1fr);gap:12px;padding:12px 18px;border-bottom:1px solid var(--line)}
.nitem i{width:10px;height:10px;border-radius:50%;margin-top:5px;background:var(--muted)}
.nitem.bad i{background:var(--bad)} .nitem.warn i{background:var(--heat)} .nitem.ok i{background:var(--ok)}
.nitem p{font-size:.88rem}
.nitem small{color:var(--muted);font-size:.76rem}
.nitem button{justify-self:start;margin-top:6px}
.nitem.new{background:var(--accent-soft)}
.toast{position:fixed;left:50%;transform:translateX(-50%);bottom:calc(18px + env(safe-area-inset-bottom,0px));z-index:30;
background:var(--ink);color:var(--bg);padding:10px 16px;border-radius:4px;font-size:.88rem;max-width:calc(100% - 32px);
box-shadow:0 6px 20px var(--shade)}

@media (max-width:900px){.tiles{grid-template-columns:repeat(3,minmax(0,1fr))}}
@media (max-width:760px){
  .sgrid{grid-template-columns:1fr}
  .stage{grid-template-columns:1fr;gap:8px}
  .tiles{grid-template-columns:repeat(2,minmax(0,1fr))}
  .tools .sep{display:none}
}
```

**public/rules.js** · 241 lines

```
/*
 * CPL-2 scheduling rules engine.
 * Pure functions with no page or server code, so the same file runs in the
 * browser (window.Rules) and in Node for the tests (require("./public/rules")).
 */
(function (root, factory) {
  const R = factory();
  if (typeof module === "object" && module.exports) module.exports = R;
  else root.Rules = R;
})(typeof self !== "undefined" ? self : this, function () {
  // NGO FP silicon grades, lowest to highest silicon, and their max Si %
  const GR = ["LSQ2L", "MSQ1", "MSQ2", "MSQ3", "HSQ1", "HSQ2", "HSQ3", "HSQ5"];
  const SI = ["0.45", "0.9", "1.3", "1.55", "1.7", "2.0", "2.2", "2.6"];

  // Default rule values. Every one of these can be changed in Rules & settings.
  const DEFAULTS = {
    thinJump: 0.8,        // max thickness jump, thinner gauge (mm)
    thickJump: 1.0,       // max thickness jump, thicker gauge (mm)
    thickFrom: 4.0,       // thicker gauge starts at (mm)
    thinWidth: 180,       // max width change, thinner gauge (mm)
    thickWidth: 300,      // max width change, thicker gauge (mm)
    thickWidthWarn: 250,  // flag ("check") thicker width change above (mm), null = off
    thkMin: 1.6, thkMax: 6.0,          // range of thickness (mm)
    startThk: null, startWdt: null,    // where auto-arrange starts, null = thinnest / widest
```

```
  sameThk: true,        // same thickness coils one after another (strict)
  critThk: 4.5,         // critical TDC rule applies above this thickness (mm)
  critTdcs: "JVPST01C00, JVPFB60AJS, JVPTR14AJS, JVPTR15AJS, JVPTR13AJS",
  ageMin: 72,           // coils must be older than this (hours)
  statuses: ["TA", "TW"],
  ngoWidth: 150,        // NGO FP max width jump (mm)
  ngoThk: null,         // NGO FP max thickness jump (mm), null = not checked
  // Row = coil rolled first, column = next coil, in GR order. Y = allowed weld.
  matrix: {
    LSQ2L: "YYYYYNNN", MSQ1: "YYYYYNNN", MSQ2: "YYYYYYNN", MSQ3: "YYYYYYYN",
    HSQ1: "YYYYYYYY", HSQ2: "YYYYYYNN", HSQ3: "NNNYYNNN", HSQ5: "NNNNYNNN"
  },
  exCust: [], exGradeHR: [], exGradeNGO: [],   // excluded customers / grades
  nBreaks: true, nHeld: true, nRemarks: true, nChanges: true, nToast: true, nBrowser: false
};

const S = v => (v == null ? "" : String(v).trim());
const N = v => { if (v === "" || v == null) return null; const x = parseFloat(v); return isNaN(x) ? null : x; };
const prodKey = o => S(o.Product).toUpperCase().replace(/\s/g, "");
const grpOf = o => { const p = prodKey(o); return p === "NGOFP" ? "NGO" : (p === "HRPO" || p === "HRSPO") ? "HR" : null; };
const gradeOf = o => (grpOf(o) === "NGO" ? S(o["Ord Crgrade"]) : S(o.Tdc));
const thkKey = o => (N(o.Thk) == null ? "" : N(o.Thk).toFixed(2));
const clone = o => JSON.parse(JSON.stringify(o));

// Fill any missing settings from DEFAULTS (keeps old saved settings working).
function withDefaults(s) {
  const m = Object.assign(clone(DEFAULTS), s || {});
  m.matrix = Object.assign(clone(DEFAULTS.matrix), (s && s.matrix) || {});
  return m;
}
const critList = set => S(set.critTdcs).split(/[\s,;]+/).filter(Boolean).map(x => x.toUpperCase());

/* ---------- Part A: data preparation (deck steps 1-14 + user filters) ---------- */
function stages(set) {
  const nEx = set.exGradeHR.length + set.exGradeNGO.length;
  return [
    { no: "1", t: "Clean the report", d: "Delete the first column and row so the column names sit in row 1.",
      r: "header row = row containing “HR Coil No”" },
    { no: "2", t: "Filter the product", d: "Keep only the products this line schedules.", r: "Product ∈ {HRPO, HRSPO, NGO FP}",
      f: o => !!grpOf(o) },
    { no: "3–4", t: "Fresh path only", d: "Act Path and Prev Unit must both be zero.", r: "Act Path = 0 AND Prev Unit = 0",
      f: o => S(o["Act Path"]) === "0" && S(o["Prev Unit"]) === "0" },
    { no: "5–6", t: "Move to the PLAN sheet", d: "Copy into the Plan sheet and delete columns not needed for scheduling.",
      r: "keep required columns" },
    { no: "7–8", t: "Status", d: "Remove TD and TU, keep only the statuses chosen in settings.",
      r: "Status ∈ {" + set.statuses.join(", ") + "}", f: o => set.statuses.includes(S(o.Status).toUpperCase()) },
    { no: "9", t: "Drop closed orders", d: "Delete every CLOSED row from Order status.", r: "Order_status ≠ CLOSED",
      f: o => S(o.Order_status).toUpperCase() !== "CLOSED" },
    { no: "10–13", t: "Split out APL7", d: "Rows with receiving remark APL7 go to the APL7 sheet.",
      r: "Receiving Remarks = APL7 → APL7 sheet", split: true },
    { no: "Filter", t: "Thickness range", d: "Coils outside the thickness range set in Rules are left out.",
      r: set.thkMin + " mm ≤ Thk ≤ " + set.thkMax + " mm", f: o => { const t = N(o.Thk);
```

```
      return t != null && t >= set.thkMin - 1e-9 && t <= set.thkMax + 1e-9; } },
    { no: "Filter", t: "Customers", d: "Customers unticked in Rules are left out.",
      r: set.exCust.length + " customer" + (set.exCust.length === 1 ? "" : "s") + " excluded", f: o => !set.exCust.includes(S(o.Cust)) },
    { no: "Filter", t: "Grades", d: "Grades (TDC for HRPO/HRSPO, grade for NGO FP) unticked in Rules are left out.",
      r: nEx + " grade" + (nEx === 1 ? "" : "s") + " excluded",
      f: o => (grpOf(o) === "NGO" ? !set.exGradeNGO.includes(gradeOf(o)) : !set.exGradeHR.includes(gradeOf(o))) },
    { no: "14", t: "Final schedule data", d: "These coils go to the HR schedule and the NGO FP report.", r: "ready to sequence" }
  ];
}

// Runs every stage in order. Returns the PLAN coils, the APL7 coils and per-stage counts.
function runPipeline(rows, set) {
  let cur = rows.slice(), apl = [];
  const steps = [];
  for (const st of stages(set)) {
    const before = cur.length; let a = 0;
    if (st.f) cur = cur.filter(st.f);
    if (st.split) {
      apl = cur.filter(o => S(o["Receiving Remarks"]) === "APL7");
      cur = cur.filter(o => S(o["Receiving Remarks"]) !== "APL7");
      a = apl.length;
    }
    steps.push({ st, before, after: cur.length, apl: a });
  }
  return {
    plan: cur, apl, steps,
    HR: cur.filter(o => grpOf(o) === "HR"),
    NGO: cur.filter(o => grpOf(o) === "NGO")
  };
}

// Remarks like "APL 7" or "APL" that the exact APL7 filter misses.
function remarkVariants(plan) {
  const v = {};
  plan.forEach(o => { const r = S(o["Receiving Remarks"]); if (r && /^apl\s*7?$/i.test(r) && r !== "APL7") v[r] = (v[r] || 0) + 1; });
  return v;
}

/* ---------- Part B/C: rules between two coils rolled one after another ---------- */
// Returns a list of [level, message, kind]; level is "bad" (break) or "warn" (check).
function pairIssues(a, b, g, set) {
  const iss = [];
  const ta = N(a.Thk), tb = N(b.Thk), wa = N(a.Wdt), wb = N(b.Wdt);
  const dt = ta != null && tb != null ? Math.abs(ta - tb) : null;
  const dw = wa != null && wb != null ? Math.abs(wa - wb) : null;
  const r2 = v => Math.round(v * 100) / 100;
  if (g === "HR") {
    if (dt != null) {
      const thin = Math.min(ta, tb) < set.thickFrom;
      const lim = thin ? set.thinJump : set.thickJump;
      if (dt > lim + 1e-9) iss.push(["bad",
        "Thickness jump " + r2(dt) + " mm > " + lim + " mm (" + (thin ? "thinner" : "thicker") + " gauge)", "t"]);
```

```
      if (dw != null) {
        const wl = thin ? set.thinWidth : set.thickWidth;
        if (dw > wl) iss.push(["bad", "Width change " + dw + " mm > " + wl + " mm (" + (thin ? "thinner" : "thicker") + " gauge)", "w"]);
        else if (!thin && set.thickWidthWarn != null && dw > set.thickWidthWarn) iss.push(["warn",
          "Width change " + dw + " mm is above " + set.thickWidthWarn + " mm", "w"]);
      }
      const c = critList(set);
      if (ta > set.critThk && tb > set.critThk && c.includes(S(a.Tdc).toUpperCase()) && c.includes(S(b.Tdc).toUpperCase()))
        iss.push(["bad", "Critical TDC back to back above " + set.critThk + " mm", "c"]);
    }
  } else {
    const ga = S(a["Ord Crgrade"]), gb = S(b["Ord Crgrade"]);
    if (!set.matrix[ga] || !set.matrix[gb]) iss.push(["warn",
      "Grade " + (!set.matrix[ga] ? ga || "blank" : gb || "blank") + " is not in the welding matrix", "g"]);
    else if (set.matrix[ga][GR.indexOf(gb)] !== "Y") iss.push(["bad", ga + " → " + gb + " is not an allowed weld", "g"]);
    if (dw != null && dw > set.ngoWidth) iss.push(["bad", "Width jump " + dw + " mm > " + set.ngoWidth + " mm", "w"]);
    if (set.ngoThk != null && dt != null && dt > set.ngoThk + 1e-9) iss.push(["bad",
      "Thickness jump " + r2(dt) + " mm > " + set.ngoThk + " mm", "t"]);
  }
  return iss;
}

const isHeld = (o, set) => { const a = N(o["Age Hours"]); return a != null && a <= set.ageMin; };

// Checks a whole sequence: each coil against the one before it, plus age and same-thickness rules.
function evaluate(g, list, set) {
  const firstAt = new Map();
  return list.map((o, i) => {
    const prev = i ? list[i - 1] : null;
    const iss = [];
    if (isHeld(o, set)) iss.push(["held", "Age " + N(o["Age Hours"]) + " h, needs more than " + set.ageMin + " h"]);
    if (prev) iss.push(...pairIssues(prev, o, g, set));
    const key = thkKey(o);
    if (set.sameThk && key && prev && thkKey(prev) !== key && firstAt.has(key))
      iss.push(["bad", key + " mm coils are split. Same thickness must run one after another (first at #" + (firstAt.get(key) + 1) + ")",
        "s"]);
    if (key && !firstAt.has(key)) firstAt.set(key, i);
    const t = N(o.Thk);
    const dt = prev && t != null && N(prev.Thk) != null ? Math.abs(t - N(prev.Thk)) : null;
    const dw = prev && N(o.Wdt) != null && N(prev.Wdt) != null ? Math.abs(N(o.Wdt) - N(prev.Wdt)) : null;
    return { o, iss, dt, dw };
  });
}

// Score for putting coil c right after coil cur: rule breaks cost the most.
function stepCost(cur, c, g, set, sameGradeBonus) {
  const iss = pairIssues(cur, c, g, set);
  const bad = iss.filter(x => x[0] === "bad").length;
  const dw = Math.abs((N(cur.Wdt) || 0) - (N(c.Wdt) || 0));
  const dt = Math.abs((N(cur.Thk) || 0) - (N(c.Thk) || 0));
  return bad * 1e7 + (iss.length - bad) * 1e5 + (sameGradeBonus && gradeOf(c) !== gradeOf(cur) ? 1e3 : 0) + dt * 1000 + dw;
}
```

```
function pickNext(cur, rem, g, set, sameGrade) {
  let bi = 0, bs = Infinity;
  rem.forEach((c, i) => { const s = stepCost(cur, c, g, set, sameGrade); if (s < bs) { bs = s; bi = i; } });
  return rem.splice(bi, 1)[0];
}

/*
 * Auto-arrange. Held coils (too fresh) go to the end.
 * With "same thickness together" on: one block per thickness, blocks visited from the
 * start thickness outward (nearest next thickness first); inside a block, same TDC/grade
 * stays together and width changes are kept small.
 */
function arrange(list, g, set) {
  const ready = list.filter(o => !isHeld(o, set)), held = list.filter(o => isHeld(o, set));
  if (!ready.length) return held;
  const st = set.startThk, sw = set.startWdt;
  const byStartWidth = (a,
    b) => (sw != null ? Math.abs((N(a.Wdt) || 0) - sw) - Math.abs((N(b.Wdt) || 0) - sw) : (N(b.Wdt) || 0) - (N(a.Wdt) || 0));

  if (set.sameThk) {
    const groups = new Map();
    ready.forEach(o => { const k = thkKey(o); if (!groups.has(k)) groups.set(k, []); groups.get(k).push(o); });
    let keys = [...groups.keys()].map(Number);
    const seq = [];
    let curT = st != null ? st : Math.min(...keys);
    while (keys.length) {
      keys.sort((a, b) => Math.abs(a - curT) - Math.abs(b - curT) || b - a);
      const k = keys.shift(); curT = k;
      const rem = groups.get(k.toFixed(2)).slice();
      if (!seq.length) { rem.sort(byStartWidth); seq.push(rem.shift()); }
      else seq.push(pickNext(seq[seq.length - 1], rem, g, set, false));
      while (rem.length) seq.push(pickNext(seq[seq.length - 1], rem, g, set, true));
    }
    return seq.concat(held);
  }

  const rem = ready.slice();
  rem.sort((a, b) => {
    if (st != null || sw != null) {
      const s = o => (st != null ? Math.abs((N(o.Thk) || 0) - st) * 1000 : 0) + (sw != null ? Math.abs((N(o.Wdt) || 0) - sw) : 0);
      return s(a) - s(b);
    }
    return (N(a.Thk) || 0) - (N(b.Thk) || 0) || (N(b.Wdt) || 0) - (N(a.Wdt) || 0);
  });
  const seq = [rem.shift()];
  while (rem.length) seq.push(pickNext(seq[seq.length - 1], rem, g, set, true));
  return seq.concat(held);
}

// Count rule breaks / checks / held coils in a sequence.
function summary(ev) {
  let breaks = 0, checks = 0, held = 0;
```

```
    ev.forEach(e => e.iss.forEach(x => { if (x[0] === "bad") breaks++; else if (x[0] === "warn") checks++; else if (x[0] === "held") held++;
      }));
    return { breaks, checks, held };
  }

  return { GR, SI, DEFAULTS, S, N, grpOf, gradeOf, thkKey, withDefaults, stages, runPipeline, remarkVariants, pairIssues, isHeld, evaluate,
    arrange, summary };
});
```

**public/app.js** · 523 lines

```
/*
 * CPL-2 Scheduler web app.
 * Uses window.Rules (rules.js) for every scheduling decision.
 * Shared state lives on the server (/api/...), with a copy in localStorage so
 * the page still opens offline or before the server answers.
 */
const R = window.Rules;
const { S, N, GR, SI, DEFAULTS, grpOf, gradeOf } = R;
const SAMPLE_NAME = "CPL2_HR_STOCK_FOR_SCHEDULING.xls";
const KEEP = ["HR Coil No", "Thk", "Wdt", "Wgt", "Age Hours", "Status", "Cust", "Tdc", "Product", "Act Path", "Prev Unit", "Order_status",
  "Receiving Remarks", "Ord Crgrade", "Silicon %", "Routing", "Edge Condth", "Next Work Center", "Coil Saddle Loc"];
const POLL_MS = 20000;

const $ = s => document.querySelector(s);
const esc = s => S(s).replace(/[&<>"]/g, c => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;" }[c]));
const f2 = v => (v == null ? "–" : String(Math.round(v * 100) / 100));
const ls = {
  get(k) { try { return JSON.parse(localStorage.getItem(k)); } catch (e) { return null; } },
  set(k, v) { try { localStorage.setItem(k, JSON.stringify(v)); } catch (e) {} }
};

/* ---------- state ---------- */
let set = R.withDefaults(ls.get("cpl2.settings"));
let stock = ls.get("cpl2.stock");
let orders = ls.get("cpl2.orders") || { HR: [], NGO: [] };
let rows = [], res = { plan: [], apl: [], steps: [], HR: [], NGO: [] };
const hideHeld = { HR: false, NGO: false };
let online = false, requiresKey = false, editorKey = ls.get("cpl2.key") || "";
let versions = ls.get("cpl2.versions") || {};
let events = ls.get("cpl2.events") || [];
let seen = ls.get("cpl2.seen") || {};
let alerts = [], lastN = {}, booted = false;

/* ---------- server API ---------- */
async function api(method, url, body) {
  const r = await fetch(url, {
    method,
    headers: Object.assign({ "Content-Type": "application/json" }, editorKey ? { "x-admin-key": editorKey } : {}),
    body: body ? JSON.stringify(body) : undefined
  });
```

```
  if (r.status === 401) { const e = new Error("key"); e.code = 401; throw e; }
  if (!r.ok) throw new Error("HTTP " + r.status);
  return r.json();
}
async function saveRemote(key, url, body) {
  try {
    const doc = await api("PUT", url, body);
    versions[key] = doc.updatedAt; ls.set("cpl2.versions", versions);
    online = true; setSync(); return true;
  } catch (e) {
    if (e.code === 401) toast("Enter the editor key in Rules & settings to save changes for everyone.");
    else { online = false; setSync(); }
    return false;
  }
}

/* ---------- pipeline ---------- */
function run() {
  res = R.runPipeline(rows, set);
  renderPrep(); renderSched("HR"); renderSched("NGO"); buildAlerts();
}
function ordered(g) {
  const list = res[g];
  const by = new Map(list.map(o => [S(o["HR Coil No"]), o]));
  const out = [], used = new Set();
  for (const id of orders[g] || []) { const o = by.get(id); if (o && !used.has(id)) { out.push(o); used.add(id); } }
  for (const o of list) { const id = S(o["HR Coil No"]); if (!used.has(id)) { out.push(o); used.add(id); } }
  return out;
}

/* ---------- schedule tab ---------- */
function renderSched(g) {
  const root = $("#p-" + g.toLowerCase() + " .wrap");
  const full = ordered(g);
  const ev = R.evaluate(g, full, set);
  const isHeld = o => R.isHeld(o, set);
  const vis = ev.filter(e => !(hideHeld[g] && isHeld(e.o)));
  const sm = R.summary(ev);
  const ton = ev.reduce((t, e) => t + (isHeld(e.o) ? 0 : N(e.o.Wgt) || 0), 0);
  $("#c" + g).textContent = full.length;
  const isHR = g === "HR";
  const applied = isHR
    ? [`Thinner gauge (&lt; ${set.thickFrom} mm): jump ≤ ${set.thinJump} mm, width ≤ ${set.thinWidth} mm`,
      `Thicker gauge (≥ ${set.thickFrom} mm): jump ≤ ${set.thickJump} mm, width ≤ ${set.thickWidth} mm`,
      `Critical TDC above ${set.critThk} mm not back to back`]
    : [`Width jump ≤ ${set.ngoWidth} mm`, "Silicon grade welding matrix",
      set.ngoThk != null ? `Thickness jump ≤ ${set.ngoThk} mm` : "Thickness jump not checked"];
  applied.push(`Age &gt; ${set.ageMin} h`);
  if (set.sameThk) applied.push("Same thickness together");
  const body = vis.map((e, i) => {
    const o = e.o;
    const bad = e.iss.some(x => x[0] === "bad"), warn = e.iss.some(x => x[0] === "warn"), h = isHeld(o);
```

```
  const cls = h ? "held" : bad ? "bad" : warn ? "warn" : "ok";
  const pill = h ? `<span class="pill info">Held</span>` : bad ? `<span class="pill bad">Break</span>` : warn ? `
    <span class="pill warn">Check</span>` : `<span class="pill ok">${i ? "OK" : "First coil"}</span>`;
  const tc = e.iss.some(x => x[2] === "t" && x[0] === "bad") ? "b" : "";
  const wc = e.iss.some(x => x[2] === "w" && x[0] === "bad") ? "b" : e.iss.some(x => x[2] === "w") ? "w" : "";
  let grpRow = "";
  const prevV = i ? vis[i - 1].o : null;
  if (set.sameThk && (!prevV || R.thkKey(prevV) !== R.thkKey(o))) {
    let n = 0, w = 0;
    for (let j = i; j < vis.length && R.thkKey(vis[j].o) === R.thkKey(o); j++) { n++; w += N(vis[j].o.Wgt) || 0; }
    grpRow = `<tr class="grp"><td colspan="15"><b>${R.thkKey(o)} mm</b><span>${n} coil${n > 1 ? "s" : ""} · ${f2(w)} t</span></td></tr>`;
  }
  return `${grpRow}<tr class="${cls}"><td class="n">${i + 1}</td>
    <td><span class="mv"><button data-g="${g}" data-mv="-1" data-i="${i}" aria-label="Move up">↑</button>
      <button data-g="${g}" data-mv="1" data-i="${i}" aria-label="Move down">↓</button></span></td>
    <td class="mono">${esc(o["HR Coil No"])}</td>${isHR ? `<td>${esc(o.Product)}</td>` : ""}
    <td class="n">${f2(N(o.Thk))}</td><td class="n">${N(o.Wdt) ?? "—"}</td>
    <td class="n dv ${tc}">${e.dt == null ? "" : f2(e.dt)}</td><td class="n dv ${wc}">${e.dw == null ? "" : e.dw}</td>
    <td class="n">${f2(N(o.Wgt))}</td><td class="n">${N(o["Age Hours"]) ?? "—"}</td>
    <td class="mono">${esc(gradeOf(o))}</td>${isHR ? "" : `<td class="n">${f2(N(o["Silicon %"]))}</td>`}
    <td class="cust" title="${esc(o.Cust)}">${esc(o.Cust)}</td><td>${esc(o.Status)}</td>
    <td><div class="why">${pill}${e.iss.map(x => `<span>${esc(x[1])}</span>`).join("")}</div></td></tr>`;
}).join("");
root.innerHTML = `
 <div class="phead"><div><div class="label">${isHR ? "HRPO + HRSPO" : "NGO FP · separate report"}</div>
   <h2>${isHR ? "HR schedule" : "NGO FP report"}</h2>
   <p>Each coil is checked against the coil rolled before it. Use Auto-arrange to build an order that follows the rules,
     then fine-tune with the arrows. The order is saved.</p></div></div>
 <div class="tiles">
   <div class="tile"><b>${full.length - sm.held}</b><span>coils ready</span></div>
   <div class="tile"><b>${f2(ton)}</b><span>tonnes ready</span></div>
   <div class="tile ${sm.breaks ? "bad" : "ok"}"><b>${sm.breaks}</b><span>rule breaks</span></div>
   <div class="tile ${sm.checks ? "warn" : "ok"}"><b>${sm.checks}</b><span>to check</span></div>
   <div class="tile"><b>${sm.held}</b><span>held, age ≤ ${set.ageMin} h</span></div>
 </div>
 <div class="applied"><span class="label" style="align-self:center">Rules applied</span>${applied.map(a => `<span class="chip">${a}
   </span>`).join("")}<button class="btn" data-goto="rules" style="padding:2px 10px">Edit</button></div>
 <div class="tools">
   <button class="btn primary" data-act="arrange" data-g="${g}">Auto-arrange</button>
   <button class="btn" data-act="sheet" data-g="${g}">Sheet order</button>
   <button class="btn" data-act="thk" data-g="${g}">Sort by thickness</button>
   <button class="btn" data-act="wdt" data-g="${g}">Sort by width</button>
   <span class="sep"></span>
   <label class="check"><input type="checkbox" id="hide-${g}" ${hideHeld[g] ? "checked" : ""}> Hide held coils</label>
 </div>
 <div class="tbl"><table class="seq"><thead><tr><th>#</th><th>Move</th><th>HR coil</th>
    ${isHR ? "<th>Product</th>" : ""}<th>Thk mm</th><th>Wdt mm</th>
    <th>Δ Thk</th><th>Δ Wdt</th><th>Wgt t</th><th>Age h</th>
    <th>${isHR ? "TDC" : "Grade"}</th>${isHR ? "" : "<th>Si %</th>"}
    <th>Customer</th><th>Status</th><th>Check vs previous coil</th></tr></thead>
 <tbody>${body || `<tr><td colspan="15" style="padding:18px;color:var(--muted)">
    No ${isHR ? "HRPO or HRSPO" : "NGO FP"} coils pass the current filters.
```

```
      Check the thickness range, customers and grades in Rules &amp;amp; settings.</td></tr>`}</tbody></table></div>`;
  root._vis = vis.map(e => S(e.o["HR Coil No"]));
}

document.addEventListener("click", e => {
  const mv = e.target.closest("button[data-mv]");
  if (mv) {
    const g = mv.dataset.g, i = +mv.dataset.i, d = +mv.dataset.mv;
    const vis = $("#p-" + g.toLowerCase() + " .wrap")._vis; const j = i + d;
    if (j < 0 || j >= vis.length) return;
    const full = ordered(g).map(o => S(o["HR Coil No"]));
    const a = full.indexOf(vis[i]), b = full.indexOf(vis[j]);
    [full[a], full[b]] = [full[b], full[a]];
    orders[g] = full; saveOrders(g); renderSched(g); buildAlerts();
    const btn = $(`#p-${g.toLowerCase()} button[data-mv="${d}"][data-i="${j}"]`); btn && btn.focus();
    return;
  }
  const act = e.target.closest("button[data-act]");
  if (act) {
    const g = act.dataset.g; let list;
    if (act.dataset.act === "arrange") list = R.arrange(res[g], g, set);
    else if (act.dataset.act === "sheet") list = res[g].slice();
    else if (act.dataset.act === "thk") list = res[g].slice().sort((a,
      b) => (N(a.Thk) ?? 0) - (N(b.Thk) ?? 0) || (N(b.Wdt) ?? 0) - (N(a.Wdt) ?? 0));
    else list = res[g].slice().sort((a, b) => (N(b.Wdt) ?? 0) - (N(a.Wdt) ?? 0) || (N(a.Thk) ?? 0) - (N(b.Thk) ?? 0));
    orders[g] = list.map(o => S(o["HR Coil No"])); saveOrders(g); renderSched(g); buildAlerts();
    toast(act.dataset.act === "arrange" ? "Coils arranged by the rules" : "Order updated");
    return;
  }
  const go = e.target.closest("[data-goto]");
  if (go) showTab(go.dataset.goto);
});
document.addEventListener("change", e => {
  const m = e.target.id && e.target.id.match(/^hide-(HR|NGO)$/);
  if (m) { hideHeld[m[1]] = e.target.checked; renderSched(m[1]); }
});

/* ---------- data prep tab ---------- */
function renderPrep() {
  const total = rows.length || 1;
  $("#funnel").innerHTML = res.steps.map(({ st, before, after, apl: a }) => {
    const drop = before - after - a; let nums;
    if (st.split) nums = `<span>${after} stay in PLAN</span><span style="color:var(--heat)">${a} → APL7</span>`;
    else if (st.f) nums = `<span>${after} of ${before}</span>
      ${drop ? `<span class="drop">−${drop}</span>` : `<span class="muted">no change
      </span>`}`;
    else nums = `<span>${after} rows</span><span class="muted">${st.no === "1" ? "start" : "no rows removed"}</span>`;
    return `<div class="stage"><div class="no">${st.no === "Filter" ? "Your filter" : "Step " + st.no}</div><div>
      <h3>${st.t}</h3><p>${st.d}</p><div class="rule">${esc(st.r)}</div></div>
     <div class="meter"><div class="barx"><i style="width:${after / total * 100}%"></i></div><div class="nums">${nums}</div></div></div>`;
  }).join("");
  const v = R.remarkVariants(res.plan); const k = Object.keys(v);
```

```
  $("#aplNote").hidden = !k.length;
  if (k.length) $("#aplNote").innerHTML = k.map(x => `<b>${v[x]}</b> coil${v[x] > 1 ? "s" : ""} with remark <code>${esc(x)}
    </code>`).join(" and ") + " stayed in PLAN because the remark is not typed exactly as APL7.";
}

/* ---------- settings tab ---------- */
const NUM = [
  { g: "jump", k: "thinJump", l: "Thinner gauge jump", u: "mm", st: 0.05, deck: "Deck: 0.4 mm in 1.6–2.0 mm" },
  { g: "jump", k: "thickJump", l: "Thicker gauge jump", u: "mm", st: 0.05, deck: "Deck: 1 mm in 2–6 mm" },
  { g: "jump", k: "thickFrom", l: "Thicker gauge starts at", u: "mm", st: 0.1,
    deck: "Coils from this thickness up use the thicker limits" },
  { g: "width", k: "thinWidth", l: "Width change, thinner gauge", u: "mm", st: 5, deck: "Deck: 150 mm below 2.0 mm" },
  { g: "width", k: "thickWidth", l: "Width change, thicker gauge", u: "mm", st: 5, deck: "Deck: 250–300 mm" },
  { g: "width", k: "thickWidthWarn", l: "Flag thicker width above", u: "mm", st: 5, deck: "Shown as Check, not a break. Blank = off",
    blank: true },
  { g: "range", k: "thkMin", l: "Thinnest coil", u: "mm", st: 0.1, deck: "Deck: 1.6 mm" },
  { g: "range", k: "thkMax", l: "Thickest coil", u: "mm", st: 0.1, deck: "Deck: 6 mm" },
  { g: "start", k: "startThk", l: "Start thickness", u: "mm", st: 0.1, deck: "Blank = start from the thinnest", blank: true },
  { g: "start", k: "startWdt", l: "Start width", u: "mm", st: 5, deck: "Blank = start from the widest", blank: true },
  { g: "crit", k: "critThk", l: "Critical TDC above", u: "mm", st: 0.1, deck: "Deck: 4.5 mm" },
  { g: "age", k: "ageMin", l: "Minimum age", u: "h", st: 1, deck: "Deck: above 72 h" },
  { g: "ngo", k: "ngoWidth", l: "NGO FP width jump", u: "mm", st: 5, deck: "Max width change between NGO FP coils" },
  { g: "ngo", k: "ngoThk", l: "NGO FP thickness jump", u: "mm", st: 0.05, deck: "Blank = not checked", blank: true }
];
const fieldsHTML = g => `<div class="fields">${NUM.filter(f => f.g === g).map(f => `<div class="field"><label for="s-${f.k}">${f.l}</label>
  <div class="inp">
  <input id="s-${f.k}" data-k="${f.k}" type="number" inputmode="decimal" step="${f.st}" ${f.blank ? 'placeholder="—"' : ""}
    value="${set[f.k] ?? ""}">
  <span>${f.u}</span></div><small id="h-${f.k}">${f.deck}</small></div>`).join("")}</div>`;
const card = (title, desc, inner, wide) => `<div class="card${wide ? " wide" : ""}"><header><h3>${title}</h3>${desc ? `<p>${desc}</p>` : ""}
  </header>${inner}</div>`;

function renderSettings() {
  const statusOpts = [...new Set(["TA", "TW", "TD", "TU", "TS", "PA", ...rows.map(o => S(o.Status).toUpperCase()).filter(Boolean)])];
  $("#settingsForm").innerHTML =
    (requiresKey ? card("Editor key", "Saving changes for everyone needs the editor key set on the server. It is kept on this device only.",
      `<div class="field"><label for="editorKey">Editor key</label><div class="inp">
      <input id="editorKey" type="password" autocomplete="current-password" value="${esc(editorKey)}"></div>
      <small>${editorKey ? "Key entered" : "Without the key you can look, but changes stay on this device"}</small></div>`, true) : "") +
    card("Thickness jump", "Largest thickness change allowed between two HRPO/HRSPO coils in a row.", fieldsHTML("jump")) +
    card("Width change", "Largest width change allowed between two HRPO/HRSPO coils in a row.", fieldsHTML("width")) +
    card("Range of thickness", "Coils outside this range are left out of both schedules.", fieldsHTML("range")) +
    card("Start of thickness and width", "Auto-arrange begins with the coil closest to these values.", fieldsHTML("start")) +
    card("Same thickness together",
      "All coils of one thickness run one after another, with the same TDC or grade kept together inside each block. " +
      "A coil that splits a thickness block counts as a rule break.",
      `<label class="check"><input type="checkbox" id="s-sameThk" ${set.sameThk ? "checked" : ""}> Same thickness one after another (strict)
      </label>`) +
    card("Critical TDC", "Above the set thickness, these TDCs must not run back to back.", fieldsHTML("crit") + `<div class="field">
      <label for="s-critTdcs">Critical TDC codes</label><textarea id="s-critTdcs" rows="2">${esc(set.critTdcs)}</textarea>
      <small>Separate codes with commas</small></div>`) +
    card("Age limit and status", "", fieldsHTML("age") + `<div class="field"><span class="label">Statuses to schedule</span>
```

```
      <div class="checks" id="s-statuses">${statusOpts.map(s => `<label>
      <input type="checkbox" value="${s}" ${set.statuses.includes(s) ? "checked" : ""}> ${s}</label>`).join("")}</div>
      <small>Deck: TA and TW only
      </small></div>`) +
    card("NGO FP report", "Rules for the separate NGO FP schedule. Click a matrix cell to switch YES / NO. Rows are the coil rolled first.",
      fieldsHTML("ngo") + `<div class="scroll"><table class="mx" id="mx"></table></div>`) +
    card("Customers", "Untick customers who are not required in this schedule.", picker("cust"), true) +
    card("Grades", "Untick grades that are not required. HRPO/HRSPO use the TDC column, NGO FP uses the order grade.",
      picker("gHR") + picker("gNGO"), true) +
    card("Notifications",
      "Choose what appears under Notifications. Phone or desktop pop-ups appear while the page is open in a background tab.",
      `
        <div class="checks">${[
          ["nBreaks", "Rule breaks in a schedule"], ["nHeld", "Coils held for age"], ["nRemarks", "APL7 remarks typed differently"],
          ["nChanges", "Rules changed on another device"], ["nToast", "Pop-up inside the page"], ["nBrowser", "Device notifications"]
        ].map(([k, l]) => `
        <label><input type="checkbox" id="s-${k}" ${set[k] ? "checked" : ""}> ${l}</label>`).join("")}</div>`, true);
  renderMatrix(); renderPickers(); markChanged();
}
const PK = {
  cust: { title: "Customer", key: "exCust", src: () => rows.filter(o => grpOf(o)), val: o => S(o.Cust) },
  gHR: { title: "HRPO / HRSPO TDC", key: "exGradeHR", src: () => rows.filter(o => grpOf(o) === "HR"), val: o => S(o.Tdc) },
  gNGO: { title: "NGO FP grade", key: "exGradeNGO", src: () => rows.filter(o => grpOf(o) === "NGO"), val: o => S(o["Ord Crgrade"]) }
};
const picker = id => `<div class="picker"><div class="bar"><span class="label">${PK[id].title}</span>
  <input type="search" id="q-${id}" placeholder="Search" aria-label="Search ${PK[id].title}">
  <button class="btn" data-pk="${id}" data-all="1">Select all</button><button class="btn" data-pk="${id}" data-all="0">Clear all</button>
  <span class="muted" id="n-${id}" style="font-size:.8rem"></span></div><div class="plist" id="l-${id}"></div></div>`;
function renderPickers() { for (const id in PK) renderPicker(id); }
function renderPicker(id) {
  const p = PK[id]; const cnt = {};
  p.src().forEach(o => { const v = p.val(o) || "(blank)"; cnt[v] = (cnt[v] || 0) + 1; });
  const q = S(($("#q-" + id) || {}).value).toLowerCase(); const ex = set[p.key];
  const keys = Object.keys(cnt).sort((a, b) => cnt[b] - cnt[a] || a.localeCompare(b));
  $("#n-" + id).textContent = `${keys.filter(k => !ex.includes(k)).length} of ${keys.length} selected`;
  $("#l-" + id).innerHTML = keys.filter(k => !q || k.toLowerCase().includes(q)).map(k => `<label class="${ex.includes(k) ? "off" : ""}">
    <input type="checkbox" data-pkv="${id}" value="${esc(k)}" ${ex.includes(k) ? "" : "checked"}><span>${esc(k)}</span>
    <em>${cnt[k]} coil${cnt[k] > 1 ? "s" : ""}</em></label>`).join("")
      || `<label><span></span><span class="muted">No match</span><span></span>
    </label>`;
  p._keys = keys;
}
function renderMatrix() {
  $("#mx").innerHTML = `<thead><tr><th>First ↓ Next →</th>${GR.map((g, i) => `<th>${g}<br><span class="muted">${SI[i]}%</span>
    </th>`).join("")}</tr></thead><tbody>${GR.map(r => `<tr><th>${r}</th>${[...set.matrix[r]].map((c, i) => `<td>
    <button data-mx="${r}" data-i="${i}" class="${c === "Y" ? "y" : "x"}"
      aria-label="${r} then ${GR[i]}: ${c === "Y" ? "allowed" : "not allowed"}">${c === "Y" ? "YES" : "NO"}
    </button></td>`).join("")}</tr>`).join("")}</tbody>`;
}
function markChanged() { NUM.forEach(f => { const h = $("#h-" + f.k); if (h) h.classList.toggle("changed", set[f.k] !== DEFAULTS[f.k]); });
  }
```

```
const form = $("#settingsForm");
form.addEventListener("input", e => {
  const t = e.target;
  if (t.id === "editorKey") { editorKey = t.value.trim(); ls.set("cpl2.key", editorKey); return; }
  if (t.dataset.k) { const f = NUM.find(x => x.k === t.dataset.k); const v = N(t.value); if (v == null && !f.blank) return; set[f.k] = v;
    markChanged(); changed(); }
  else if (t.id === "s-critTdcs") { set.critTdcs = t.value; changed(); }
  else if (t.id && t.id.startsWith("q-")) renderPicker(t.id.slice(2));
});
form.addEventListener("change", async e => {
  const t = e.target;
  if (t.id === "editorKey") { if (editorKey) { await saveSettings(); toast("Editor key saved on this device"); } return; }
  if (t.closest("#s-statuses")) { set.statuses = [...document.querySelectorAll("#s-statuses input:checked")].map(i => i.value); changed(); }
  else if (t.dataset.pkv) {
    const p = PK[t.dataset.pkv]; const ex = new Set(set[p.key]);
    t.checked ? ex.delete(t.value) : ex.add(t.value); set[p.key] = [...ex];
    t.closest("label").classList.toggle("off", !t.checked);
    const keys = p._keys || []; $("#n-" + t.dataset.pkv).textContent = `${keys.filter(k => !ex.has(k)).length} of ${keys.length} selected`;
    changed();
  }
  else if (t.id === "s-nBrowser") {
    if (t.checked) {
      if (!("Notification" in window)) { t.checked = false; toast("This browser does not support device notifications."); return; }
      const p = await Notification.requestPermission();
      if (p !== "granted") { t.checked = false; toast("Notifications are blocked. Allow them in the browser's site settings."); return; }
    }
    set.nBrowser = t.checked; changed();
  }
  else if (t.id && t.id.startsWith("s-") && t.type === "checkbox") { set[t.id.slice(2)] = t.checked; changed(); }
});
form.addEventListener("click", e => {
  const b = e.target.closest("button[data-mx]");
  if (b) {
    const r = b.dataset.mx, i = +b.dataset.i; const a = [...set.matrix[r]];
    a[i] = a[i] === "Y" ? "N" : "Y"; set.matrix[r] = a.join("");
    renderMatrix(); changed();
    const nb = $(`button[data-mx="${r}"][data-i="${i}"]`); nb && nb.focus();
    return;
  }
  const pk = e.target.closest("button[data-pk]");
  if (pk) { const p = PK[pk.dataset.pk]; set[p.key] = pk.dataset.all === "1" ? [] : (p._keys || []).slice(); renderPicker(pk.dataset.pk);
    changed(); }
});
let resetArm = 0;
$("#resetBtn").addEventListener("click", () => {
  if (Date.now() - resetArm > 4000) {
    resetArm = Date.now(); $("#resetBtn").textContent = "Click again to reset";
    setTimeout(() => ($("#resetBtn").textContent = "Reset to default rules"), 4000); return;
  }
  resetArm = 0; $("#resetBtn").textContent = "Reset to default rules";
  set = R.withDefaults({}); renderSettings(); changed(); toast("Rules reset to defaults");
});
```

```
/* ---------- saving ---------- */
let saveT = null;
function changed() { run(); setSave("pending"); clearTimeout(saveT); saveT = setTimeout(saveSettings, 700); }
async function saveSettings() {
  ls.set("cpl2.settings", set);
  const ok = await saveRemote("settings", "/api/settings", set);
  setSave(ok ? "ok" : "local");
}
function setSave(s) {
  const el = $("#saveState");
  const t = { pending: "Saving…", ok: "All changes saved for every device", local: "Saved on this device only" }[s];
  el.className = "sync " + (s === "ok" ? "on" : s === "local" ? "local" : "");
  el.querySelector("span").textContent = t;
}
function setSync() {
  const el = $("#sync");
  el.className = "sync " + (online ? "on" : "local");
  el.querySelector("span").textContent = online ? "Synced across devices" : "Offline: saved on this device";
}
const ordT = {};
function saveOrders(g) {
  ls.set("cpl2.orders", orders);
  clearTimeout(ordT[g]);
  ordT[g] = setTimeout(() => saveRemote("plan-" + g, "/api/plan/" + g, { ids: orders[g] }), 800);
}
async function saveStock() {
  ls.set("cpl2.stock", stock);
  const ok = await saveRemote("stock", "/api/stock", stock);
  if (!ok) toast("The file is loaded on this device. It could not be shared with other devices.");
}

/* ---------- loading a stock file ---------- */
function loadAOA(aoa, name) {
  const hi = aoa.findIndex(r => r && r.some(c => S(c) === "HR Coil No"));
  if (hi < 0) { toast("No “HR Coil No” column found. Load the Hrstock Report export."); return; }
  const head = aoa[hi].map(S); const idx = KEEP.map(c => head.indexOf(c));
  const missing = KEEP.filter((c, i) => idx[i] < 0);
  const data = aoa.slice(hi + 1).filter(r => r && r.some(c => c != null && c !== "")).map(r => idx.map(i => {
    if (i < 0) return null; const v = r[i];
    return typeof v === "number" ? Math.round(v * 1000) / 1000 : v == null ? null : S(v);
  }));
  stock = { name, cols: KEEP, rows: data }; useStock(); saveStock();
  pushEvent("ok", `Loaded ${name}: ${data.length} rows` + (missing.length ? `. Missing columns: ${missing.join(", ")}` : ""));
  toast(`${data.length} rows loaded from ${name}`);
}
$("#file").addEventListener("change", async e => {
  const f = e.target.files[0]; if (!f) return;
  if (typeof XLSX === "undefined") { toast("The spreadsheet reader did not load. Check the connection and reload."); return; }
  try {
    const wb = XLSX.read(await f.arrayBuffer(), { type: "array" });
    const sh = wb.Sheets[wb.SheetNames.find(n => /hrstock/i.test(n)) || wb.SheetNames[0]];
    loadAOA(XLSX.utils.sheet_to_json(sh, { header: 1, raw: true, defval: null }), f.name);
```

```
  } catch (err) { toast("That file could not be read. Use the .xls or .xlsx export of the HR stock report."); }
  e.target.value = "";
});
function useStock() {
  rows = stock.rows.map(r => { const o = {}; stock.cols.forEach((c, i) => (o[c] = r[i])); return o; });
  $("#srcName").textContent = stock.name; renderSettings(); run();
}

/* ---------- notifications ---------- */
function pushEvent(level, text) {
  events.unshift({ id: "e" + Date.now(), level, text, at: Date.now() });
  events = events.slice(0, 20); ls.set("cpl2.events", events); buildAlerts();
}
function buildAlerts() {
  const a = [];
  for (const g of ["HR", "NGO"]) {
    const ev = R.evaluate(g, ordered(g), set); const sm = R.summary(ev);
    const name = g === "HR" ? "HR schedule" : "NGO FP report";
    if (set.nBreaks && sm.breaks) a.push({ key: `br-${g}-${sm.breaks}`, level: "bad",
      text: `${sm.breaks} rule break${sm.breaks > 1 ? "s" : ""} in the ${name}.`, go: g.toLowerCase(), btn: "Open " + name });
    if (set.nHeld && sm.held) a.push({ key: `hd-${g}-${sm.held}`, level: "warn",
      text: `${sm.held} coil${sm.held > 1 ? "s" : ""} in the ${name} ${sm.held > 1 ? "are" : "is"} ${set.ageMin} h old or less and held.`,
      go: g.toLowerCase(), btn: "Open " + name });
  }
  const v = R.remarkVariants(res.plan); const n = Object.values(v).reduce((x, y) => x + y, 0);
  if (set.nRemarks && n) a.push({ key: `rm-${n}`, level: "warn",
    text: `${n} coil${n > 1 ? "s have" : " has"} receiving remark ${Object.keys(v).map(k => "“" + k + "”").join(" / ")}` +
      " instead of APL7 and stayed in PLAN.",
    go: "prep", btn: "Open data prep" });
  if (!a.length) a.push({ key: "clear", level: "ok", text: "No open alerts. Both schedules follow the current rules." });
  const ev = events.filter(e => set.nChanges || !e.text.startsWith("Rules changed")).map(e => ({ key: e.id, level: e.level, text: e.text,
    at: e.at }));
  const prev = new Set(alerts.map(x => x.key));
  alerts = a.concat(ev);
  const unseen = alerts.filter(x => !seen[x.key] && x.key !== "clear");
  $("#bellCount").hidden = !unseen.length; $("#bellCount").textContent = unseen.length;
  const kindOf = k => k.replace(/-\d+$/, ""), numOf = k => +k.split("-").pop() || 0;
  const fresh = alerts.filter(x => x.key !== "clear" && !seen[x.key] && !prev.has(x.key) &&
    (x.at || numOf(x.key) > (lastN[kindOf(x.key)] || 0)));
  lastN = {}; a.forEach(x => (lastN[kindOf(x.key)] = numOf(x.key)));
  if (booted && fresh.length) {
    if (set.nToast) toast(fresh[0].text);
    if (set.nBrowser && document.hidden && "Notification" in window && Notification.permission === "granted") {
      try { new Notification("CPL-2 Scheduler", { body: fresh[0].text, tag: kindOf(fresh[0].key) }); } catch (e) {}
    }
  }
  if (!$("#drawer").hidden) renderDrawer();
}
function renderDrawer() {
  $("#nlist").innerHTML = alerts.map(x => `<div class="nitem ${x.level}${!seen[x.key] && x.key !== "clear" ? " new" : ""}"><i></i><div>
    <p>${esc(x.text)}</p>${x.at ? `<small>${new Date(x.at).toLocaleString()}</small>` : "<small>Current</small>"}${x.go ? `<br>
    <button class="btn" data-goto="${x.go}">${x.btn}</button>` : ""}</div></div>`).join("");
```

```
}
function openDrawer() { $("#drawer").hidden = false; $("#shade").hidden = false; renderDrawer(); $("#closeDrawer").focus(); }
function closeDrawer() { alerts.forEach(x => (seen[x.key] = 1)); ls.set("cpl2.seen", seen); $("#drawer").hidden = true;
  $("#shade").hidden = true; buildAlerts(); }
$("#bellBtn").onclick = openDrawer; $("#closeDrawer").onclick = closeDrawer; $("#shade").onclick = closeDrawer;
$("#markRead").onclick = () => { alerts.forEach(x => (seen[x.key] = 1)); ls.set("cpl2.seen", seen); renderDrawer(); buildAlerts(); };
$("#nlist").addEventListener("click", e => { if (e.target.closest("[data-goto]")) closeDrawer(); });
document.addEventListener("keydown", e => { if (e.key === "Escape" && !$("#drawer").hidden) closeDrawer(); });
let toastT;
function toast(t) { const el = $("#toast"); el.textContent = t; el.hidden = false; clearTimeout(toastT);
  toastT = setTimeout(() => (el.hidden = true), 3600); }

/* ---------- tabs ---------- */
function showTab(id) {
  document.querySelectorAll("nav.tabs button").forEach(b => b.setAttribute("aria-selected", b.dataset.tab === id));
  ["hr", "ngo", "rules", "prep"].forEach(t => ($("#p-" + t).hidden = t !== id));
  ls.set("cpl2.tab", id); history.replaceState(null, "", "#" + id); window.scrollTo({ top: 0 });
}
document.querySelectorAll("nav.tabs button").forEach(b => (b.onclick = () => showTab(b.dataset.tab)));

/* ---------- sync with the server ---------- */
// Applies anything on the server that is newer than what this device last saw.
async function pull(first) {
  let body;
  try { body = await api("GET", "/api/state"); online = true; } catch (e) { online = false; setSync(); return; }
  const wasKey = requiresKey; requiresKey = !!body.requiresKey;
  const st = body.state || {};
  const newer = k => st[k] && st[k].updatedAt !== versions[k];
  let rerun = false;
  if (newer("settings")) {
    set = R.withDefaults(st.settings.data); ls.set("cpl2.settings", set);
    if (!first) pushEvent("warn", "Rules changed on another device.");
    rerun = true;
  } else if (first && !st.settings && ls.get("cpl2.settings") && (!requiresKey || editorKey)) {
    saveSettings(); // first run against an empty server: share this device's settings
  }
  if (newer("stock")) {
    stock = st.stock.data; ls.set("cpl2.stock", stock);
    if (!first) pushEvent("ok", `New stock file loaded on another device: ${stock.name}`);
    rerun = true;
  }
  for (const g of ["HR", "NGO"]) {
    if (newer("plan-" + g)) { orders[g] = st["plan-" + g].data.ids || []; ls.set("cpl2.orders", orders); rerun = true; }
  }
  for (const k of Object.keys(st)) versions[k] = st[k].updatedAt;
  ls.set("cpl2.versions", versions);
  if (rerun || wasKey !== requiresKey) useStock();
  setSync();
}

/* ---------- start ---------- */
(async function boot() {
```

```
  if (!stock) {
    try { const r = await fetch("sample-stock.json"); const j = await r.json(); stock = { name: SAMPLE_NAME, cols: j.cols, rows: j.rows }; }
    catch (e) { stock = { name: "No stock loaded", cols: KEEP, rows: [] }; }
  }
  useStock();
  const start = location.hash.slice(1);
  showTab(["hr", "ngo", "rules", "prep"].includes(start) ? start : ls.get("cpl2.tab") || "hr");
  setSync();
  await pull(true);
  booted = true;
  setInterval(() => pull(false), POLL_MS); // keeps polling in a background tab so device notifications can fire
  document.addEventListener("visibilitychange", () => { if (!document.hidden) pull(false); });
})();
```

**test/rules.test.js** · 62 lines

```
// Run with: npm test
const test = require("node:test");
const assert = require("node:assert");
const R = require("../public/rules");
const sample = require("../public/sample-stock.json");

const rows = sample.rows.map(r => Object.fromEntries(sample.cols.map((c, i) => [c, r[i]])));
const set = R.withDefaults({});
const coil = (thk, wdt, extra = {}) =>
  Object.assign({ "HR Coil No": String(Math.random()), Thk: thk, Wdt: wdt, Product: "HRPO", "Age Hours": 100, Tdc: "X" }, extra);

test("data prep matches the deck steps on the sample file", () => {
  const res = R.runPipeline(rows, set);
  assert.strictEqual(rows.length, 270);
  assert.strictEqual(res.apl.length, 60);
  assert.strictEqual(res.plan.length, 143); // 144 after step 13, minus one 6.5 mm coil outside 1.6-6 mm
});

test("thinner gauge thickness jump limit is 0.8 mm", () => {
  assert.strictEqual(R.pairIssues(coil(2.0, 1200), coil(2.8, 1200), "HR", set).length, 0);
  const iss = R.pairIssues(coil(2.0, 1200), coil(2.9, 1200), "HR", set);
  assert.ok(iss.some(x => x[0] === "bad" && x[2] === "t"));
});

test("thicker gauge (4 mm and above) thickness jump limit is 1 mm", () => {
  assert.strictEqual(R.pairIssues(coil(4.0, 1200), coil(5.0, 1200), "HR", set).length, 0);
  assert.ok(R.pairIssues(coil(4.0, 1200), coil(5.2, 1200), "HR", set).some(x => x[2] === "t"));
});

test("thinner gauge width change limit is 180 mm", () => {
  assert.strictEqual(R.pairIssues(coil(2, 1200), coil(2, 1380), "HR", set).length, 0);
  assert.ok(R.pairIssues(coil(2, 1200), coil(2, 1390), "HR", set).some(x => x[2] === "w"));
});

test("critical TDCs above 4.5 mm cannot run back to back", () => {
```

```
  const a = coil(5, 1200, { Tdc: "JVPST01C00" }), b = coil(5, 1200, { Tdc: "JVPTR14AJS" });
  assert.ok(R.pairIssues(a, b, "HR", set).some(x => x[2] === "c"));
});

test("NGO FP uses the welding matrix and the 150 mm width jump", () => {
  const n = (g, w) => ({ Thk: 2.6, Wdt: w, Product: "NGO FP", "Ord Crgrade": g });
  assert.ok(R.pairIssues(n("LSQ2L", 1000), n("HSQ2", 1000), "NGO", set).some(x => x[2] === "g"));
  assert.strictEqual(R.pairIssues(n("HSQ2", 1000), n("LSQ2L", 1000), "NGO", set).length, 0);
  assert.ok(R.pairIssues(n("LSQ2L", 1000), n("LSQ2L", 1160), "NGO", set).some(x => x[2] === "w"));
});

test("auto-arrange keeps every thickness in one block", () => {
  const res = R.runPipeline(rows, set);
  const seq = R.arrange(res.HR, "HR", set);
  assert.strictEqual(seq.length, res.HR.length);
  const keys = seq.map(R.thkKey);
  const blocks = keys.filter((k, i) => i === 0 || k !== keys[i - 1]);
  assert.strictEqual(blocks.length, new Set(keys).size);
  const ev = R.evaluate("HR", seq, set);
  assert.ok(!ev.some(e => e.iss.some(x => x[2] === "s")), "no split thickness blocks");
});

test("changed settings change the result", () => {
  const strict = R.withDefaults({ thinWidth: 50 });
  assert.ok(R.pairIssues(coil(2, 1200), coil(2, 1260), "HR", strict).some(x => x[2] === "w"));
});
```

**README.md** · 42 lines

```
# CPL-2 Scheduler

Coil scheduling rulebook and scheduler for the CPL-2 line (HRPO / HRSPO and NGO FP).

- Runs the deck's data-prep steps on an HR stock report (.xls / .xlsx)
- Checks every coil against the one before it using editable rules
- Auto-arranges both schedules (same thickness together)
- Settings, stock file and orders are shared across devices
- In-page and device notifications

## Run locally

```bash
npm install
cp .env.example .env     # optional
npm start                # http://localhost:3000
npm test                 # rules engine tests
```

Without `MONGODB_URI` the app stores data in `data/state.json`.

## Deploy (Render + MongoDB Atlas, free)
```

````
1. Create a free M0 cluster on MongoDB Atlas, a database user, and allow `0.0.0.0/0` under Network Access.
2. Push this folder to a GitHub repository.
3. On Render: New > Web Service > connect the repo.
   Build command `npm install`, start command `npm start`, instance type Free.
4. Add environment variables `MONGODB_URI` (Atlas connection string) and `ADMIN_KEY` (any secret).
5. Open the `.onrender.com` link. Enter the editor key in Rules & settings once per device.

## API

| Method | Path | Body |
| --- | --- | --- |
| GET | `/api/health` | |
| GET | `/api/state` | |
| PUT | `/api/settings` | settings object |
| PUT | `/api/stock` | `{ name, cols, rows }` |
| PUT | `/api/plan/HR` or `/api/plan/NGO` | `{ ids: [...] }` |

PUT requests need the header `x-admin-key` when `ADMIN_KEY` is set.
````